

ERNN: Error-Resilient RNN for Encrypted Traffic Detection towards Network-Induced Phenomena

Ziming Zhao, Zhaoxuan Li, Jialun Jiang, Fengyuan Yu, Fan Zhang[✉], *Member, IEEE*,
Congyuan Xu[✉], Xinjie Zhao, Rui Zhang, Shize Guo

Abstract—Traffic detection systems based on machine learning have been proposed to defend against cybersecurity threats, such as intrusion attacks and malware. However, they did not take the impact of network-induced phenomena into consideration, such as packet loss, retransmission, and out-of-order. These phenomena will introduce additional misclassifications in the real world. In this paper, we present ERNN, a robust and end-to-end RNN model that is specially designed against network-induced phenomena. As its core, ERNN is designed with a novel gating unit named as session gate that includes: (i) four types of actions to simulate common network-induced phenomena during model training; and (ii) the Mealy machine to update states of session gate that adjusts the probability distribution of network-induced phenomena. Taken together, ERNN advances state-of-the-art by realizing the model robustness for network-induced phenomena in an error-resilient manner. We implement ERNN and evaluate it extensively on both intrusion detection and malware detection systems. By practical evaluation with dynamic bandwidth utilization and different network topologies, we demonstrate that ERNN can still identify 98.63% of encrypted intrusion traffic when facing about 16% abnormal packet sequences on a 10 Gbps dataplane. Similarly, ERNN can still robustly identify more than 97% of the encrypted malware traffic in multi-user concurrency scenarios. ERNN can realize >4% accuracy more than SOTA methods. Based on the Integrated Gradients method, we interpret the gating mechanism can reduce the dependencies on local packets (termed *dependency dispersion*). Moreover, we demonstrate that ERNN possesses superior stability and scalability in terms of parameter settings and feature selection.

Index Terms—Network-induced phenomena, network intrusion/malware detection systems, wired/wireless network, model robustness



1 INTRODUCTION

TRAFFIC detection systems as an important part of the security mechanisms for discovering on-going malicious activities in networks [1], [2]. The typical detection approach aims to describe the special pattern for each type of attack, so it often maintains a library of signatures for known attacks. The effectiveness of these policy-based detection systems [3]–[7] largely depends on the knowledge completeness of attack patterns. After the classic policy-based methods, machine-learning-based (ML-based in short) detection systems emerged and it has raised much attention from both academia and industrial communities [8]. In these systems, sequence-based machine learning models are widely used, such as Markov chains [9]–[12] and recurrent neural network (RNN) models [13]. Current works generally train and evaluate their models based on proxy logs [14]

collected by the packet capturer [15] (such as *tshark* [16]). They achieved high accuracy in a laboratory environment.

However, complex network topology, such as distributed systems, inevitably produces network-induced phenomena [17], [18]. Such network-induced phenomena include but are not limited to transmission delays, packet dropouts, missing/fading measurements, out-of-order packets, and variable sampling/transmission intervals [19], [20]. It not only deteriorates the performance of network systems but also introduces the difference between packet capturer and execution server [15], [21]. For instance, out-of-order TCP packets due to delay are directly collected by the capturer but will be reordered [22], [23] by the TCP server at the application layer. By measuring real-world HTTP servers with the probe packet, previous work reveals the existence of persistent and high reordering rates (e.g., 10%) on network links [24]. Therefore, we should consider the influence of network-induced phenomena on detection systems [14].

Prior works [25]–[30] proposed a series of additional methods to solve network-induced phenomena. For instance, He et al. [30] utilized a dedicated reordering buffer under the transport layer to deal with out-of-order packets. While Lee et al. [29] stored the original data packet for comparison to find the TCP retransmitted data packets. However, their methods need to perform extra preprocessing before training the machine learning model [29]. Also, they require to design complicated extensions for other protocols such as KCP, and lack applicability for UDP protocol. Additionally, we summarize the traffic recovery capabilities of 11 popular traffic analysis tools in § 2.4.

Furthermore, there arise many studies about model

- Z. Zhao, J. Jiang, F. Yu, F. Zhang, X. Zhao, and S. Guo are with Zhejiang University, Hangzhou, 310027, China. F. Zhang is also with ZJU-Hangzhou Global Scientific and Technological Innovation Center, 311200, with the Key Laboratory of Blockchain and Cyberspace Governance of Zhejiang Province, 310027 and with Jiaxing Research Institute, Zhejiang University, 314000, China. E-mail: {zhaoziming, jgallon, fengyuanyu, fanzhang}@zju.edu.cn, zhaoxinjieem@163.com, nsfsgsz@126.com.
- C. Xu is with Jiaxing University College of Information Science and Engineering, Jiaxing, Zhejiang, 314001, China, and Jiaxing Research Institute, Zhejiang University, Hangzhou, 310027, China. E-mail: cyxu@zju.edu.cn.
- Z. Li and R. Zhang are with the State Key Laboratory of Information Security, Institute of Information Engineering, CAS, Beijing, 100093, China, and also with the School of Cyber Security, UCAS, Beijing, 100049, China. E-mail: {lizhaoxuan, zhangrui}@iie.ac.cn.

Manuscript received February 17, 2022; revised 18 Nov, 2022. (Corresponding authors: Fan Zhang and Congyuan Xu.)

robustness in detection systems. As some leading work, Pascoal et al. [31] used mutual information metric to design robust statistics features. Xie et al. [32] designed subspace clustering to identify the traffic of each application in isolation. Li et al. [33] proposed the sub-linear classifier to learn against malicious feature manipulation with delayed feedback. However, our focus is different from the above literatures. On the one hand, their studies center around statistics-based features (*e.g.*, average packet length) rather than sequence-based methods. Considering that existing sequence-based models have been verified can extract fine-grained traffic semantics [13], [15], [34] (*e.g.*, detecting evasion packets), it is meaningful to study the robust sequence-based scheme. On the other hand, their models do not consider the network-induced phenomena which are universal in real-world scenarios.

Therefore, fully realizing the sequence-based learning capability in detection systems is an open problem. The challenging nature stems from meeting the following two requirements.

(i) *End-to-end learning*. It is complicated to design the reordering algorithm and delete retransmission packets for each protocol. Moreover, it is difficult to find the lost packets. More discussion on the limitations of preprocessing-manner traffic normalization tools can be found in § 2.4. Therefore, an end-to-end model needs to be designed which does not require additional packet reordering during the data preprocessing. And this model does not rely on specific network protocols.

(ii) *Universal adaptability*. A more crucial problem is to enable the model to adapt to different network scenarios. Previous works [35]–[37] confirmed that bandwidth and topology structures are the main reasons affecting the network quality-of-service (QoS). On the one hand, high bandwidth utilization will cause network-induced phenomena. For example, the TCP burst will cause a lot of packet loss due to buffer queue overflow. On the other hand, the complex network topology may increase the network-induced phenomena. For instance, the existence of parallel links may introduce a large number of out-of-order packets.

In this paper, we formulate the network-induced phenomena and investigate their impact on sequence-based detection systems. Towards complex real-world networks, we present ERNN, a robust and end-to-end model to address this challenging problem in an error-resilient manner. ERNN is designed with a gating unit named as **session gate** on traditional RNN (*e.g.*, Long Short-Term Memory, LSTM in short), for shaping traffic sequence features. During model training, ERNN reduces dependence on an individual packet by introducing various actions and state transitions.

Our contributions are summarized as follows:

- We formulate the network-induced phenomena, such as packet loss, retransmission, and out-of-order. Then we study its influence on existing traffic detection systems. The results show that the network-induced phenomena will introduce a large number of misclassification in current state-of-the-art (SOTA) methods.
- We present ERNN, an error-resilient and end-to-end RNN model that can adapt to complex networks. ERNN is designed with a novel gating unit named as **session gate** for enhancing the model robustness.

During model training, **session gate** provides four types of actions to simulate common network-induced phenomena. Between RNN units on the timeline, we use the Mealy machine to update the state of **session gate** that adjusts the probability distribution of network-induced phenomena.

- We evaluate ERNN substantially on both wired and wireless physical testbeds. By practical evaluation with dynamic bandwidth utilization and different network topologies, we demonstrate that ERNN can work well against the network-induced phenomenon in an error-resilient manner. Specifically, ERNN can still identify 98.63% of encrypted intrusion traffic when facing about 16% abnormal packet sequences on a 10 Gbps wired dataplane. Similarly, in wireless scenarios with different degrees of abnormal packet sequences caused by multi-user concurrency, ERNN can still robustly identify more than 97% of the encrypted malware traffic.
- We interpret the gating mechanism based on the Integrated Gradients method. Our results show that ERNN reduces the dependencies on local packets (termed *dependency dispersion*). Moreover, we demonstrate that ERNN possesses remarkable stability and scalability in terms of parameter settings and feature selection.

2 RELATED WORK

In this section, we outline the current work with respect to cybersecurity threats detection, existing robust approaches, and traffic normalization tools.

2.1 Conventional Threats Detection in Cybersecurity

Previous research approaches on cybersecurity threats detection can be summarized as follow. The port-based method [38] utilized the port list provided by Internet Assigned Numbers Administration (IANA) to identify attack types at protocol levels, but it did not adapt to check the dynamic ports. The payload-based method [39] tried to search and match specific signatures in the payload of the packet. Moreover, Sen et al. [40] provided a protocol-based method to detect P2P traffic with application load features. However, the above methods lose their efficiency in encrypted traffic detection because it is impossible to get signatures from the payload after encryption.

2.2 ML-based Threats Detection in Cybersecurity

(i) *Statistics-based methods*. Recent work adopted machine learning (ML) methods to identify malicious behaviors. Some statistics-based methods [41], [42] aimed to design a set of statistical features and applied ML models (such as random forests) for detection. Taylor et al. [41] presented ML models (such as support vector machines) to identify smartphone Apps by traffic analysis. Some ML-based network intrusion detection systems embraced the advances in machine learning to identify attack traffic. For example, Riyami et al. [43] proposed an alternative strategy that aims to evaluate the performance of malicious traffic detection models. Mirsky et al. [15] proposed to use AutoEncoder to detect abnormal behavior from network traffic.

TABLE 1
The traffic normalization capabilities of the popular preprocessing-manner tools.

ID	Supported Layer 5-7 Protocol	Layer 4 Protocol	Tools	Rearrangement (Out-of-order)	Discard (Retransmission)	Recovery (Loss)	Visualization
①	FTP, Gopher, HTTP, IMAP4, IRC, NNTP, XMPP, POP3, SMTP, SSH, TELNET, RPC, TLS, BGP	TCP, SCTP, ...	WireEdit	Manual modification		no	Window
			Wireshark	①\③\④	①\③\④	no	Window
			Tcpdump	no	①\③\④	no	Command
②	DHCP, DNS, SNMP, RTP/RTCP, SDP, NTP, SSDP, TFTP, NFS, RIP	UDP, DCCP, ...	Sortpcap	①\③\④	no	no	Command
			Tcptrace	no	①\③\④	no	Command
③	KCP, ... (custom protocols with special markers) QUIC, ... (improved layer-4 protocols)		QpA	no	no	no	Window
			Istat	①\③\④	no	no	Web
④	SIP, RTSP, SOAP, GTP, STUN	TCP/UDP	CapAnalysis	no	no	no	Web
			Xplico	no	no	no	Web
⑤	Others: ICMP, IGMP, IS-IS, ARP, RARP, NAT, RSVP, RSVP, OSPF, ...		OMA	①\③\④	①\③\④	no	Window
			Moloch	no	no	no	Window

(ii) *Sequence-based methods*. Meanwhile, it emerged some sequence-based methods in this field. Maciej et al. [9] encoded the SSL/TLS protocol (such as handshake) and provided FoSM which employed the first-order Markov model to classify multiple representative applications. Subsequently, Liu et al. [10] introduced the packet length sequence and proposed MaMPF that used the transition probability matrix of the Markov model as the input for machine learning models (*e.g.*, random forests). Shen et al. [11] proposed SOCRT by replacing FoSM with a second-order Markov model and combining the length of the certificate packet. Based on SOCRT, they [12] proposed SOB to use the length of the certificate packet and the length of the first communication packet. Then, FS-Net [13] used the bi-GRU model to learn the sequence features and classify 18 common applications.

2.3 Current Robust Detection Approach

There arise many studies about model robustness in threats detection for encrypted traffic [31]–[33]. Specifically, Pascoal et al. [31] used mutual information metrics to design robust statistics features. Xie et al. [32] designed subspace clustering to identify the traffic of each class in isolation.

As a recent art, Li et al. [33] advanced the state by considering robust learning against malicious data. They studied the problem of malicious feature manipulation with delayed feedback and designed a classifier to detect malicious traffic. Thus, despite the similar purpose for robustness, ERNN is fundamentally different from it in the following three aspects: (i) We focus on sequence-based methods and analyze the more common network-induced phenomenon, such as packet loss, retransmission, and out-of-order. (ii) We evaluate the influence of network-induced phenomenon on the sequence-based SOTA model in encrypted traffic detection. (iii) Our error-resilient design is widely applicable and embraces the advances of powerful deep neural networks. Moreover, Whisper [44] focuses on the robustness against adversaries that inject benign packets into malicious traffic. We will compare with Whisper in the malware detection experiment in § 6.2 (since it is binary classification) and explore its robustness against the network-induced phenomena.

2.4 Preprocessing-Manner Traffic Normalization

Readers could concern that whether we can perform traffic normalization in the preprocessing-manner before feeding models, thereby eliminating the effects of network-induced phenomena. We develop a comprehensive investigation

from the aspects of tools and corresponding protocols, and summarize them in Table 1.

In the left part, we outline a series of common layer 5~7 protocols that can be roughly categorized into five types based on the corresponding transport layer protocol. There are: ① the layer-4 protocol is based on TCP, SCTP, etc.; ② the layer-4 protocol is based on UDP, DCCP, etc.; ③ some customized protocols such as KCP, QUIC; ④ the layer-4 protocol can be based on either TCP or UDP, *e.g.*, SIP, RTSP; ⑤ some other protocols, *e.g.*, ICMP, ARP. This division can facilitate the subsequent discussion about the traffic repair capability of different tools.

For the right part, we examine the traffic normalization capabilities of 11 popular tools in terms of rearrangement (for out-of-order packets), discard (for retransmitted packets), and recovery (for lost packets). Given the existence of the TCP sequence number or other special markers (*e.g.*, KCP protocol adds the session, message, and fragmentation numbers to UDP), it provides feasibility for rearranging out-of-order packets and discarding retransmission packets. Therefore, these three types of protocols (*i.e.*, ①\③\④) are usually supported simultaneously. In general, a tool such as *Wireshark* holds both capabilities of rearrangement and discard. While some tools only support one endogenous capability, such as *Tcpdump* could discard the retransmitted packets. Other tools (*e.g.*, *WireEdit*) may need to be secondarily developed by practitioners to cope with specific requirements. Particularly, all these tools could not recover the lost packets due to lacking the original packets on the sender side.

Overall, existing tools are limited in their ability to normalize traffic. For instance, they cannot recover the lost packets and unable to repair those connectionless packets (*e.g.*, UDP-based protocols). This indicates that research on error-resilient models is valuable, meanwhile, § 7.3 conducts a series of evaluations that combine traffic normalization tools with models.

3 ASSUMPTIONS AND PROBLEM FORMULATION

In this section, we formalize the network-induced phenomena and describe our adversary model.

3.1 Threat Model and Assumptions

The threat model that we consider is depicted in Fig. 1. We assume that the traffic capturer is located in the server host, and is capable of reading all the packets received. Particularly, it can also be deployed in the network switch which catches the packets exchanged between the client and server.

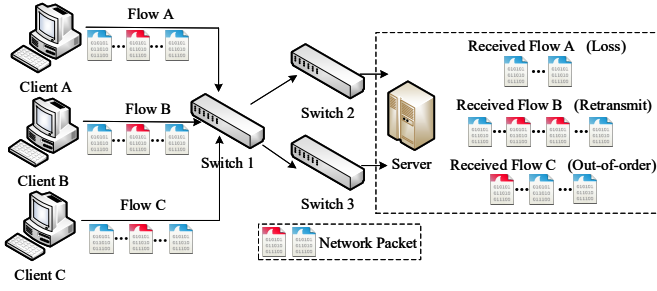


Fig. 1. Threat model with network-induced phenomena.

We mainly consider three types of network-induced phenomena, *i.e.*, packet loss, retransmission, and out-of-order. In Fig. 1, as concrete cases, Flow A, B, and C represent packet loss, retransmission, and out-of-order, respectively. Moreover, the situation of multiple phenomena that appear simultaneously will also be studied. For example, when a packet is lost and not retransmitted in time, subsequent packets received will induce out-of-order phenomena.

3.2 Definition of Network-induced Phenomena

For three basic network-induced phenomena, we provide here precise definitions of them used in this work, as well as the problem that ERNN will solve. First, we clarify the related notation about the basic flow unit (5-tuple).

Flow Formulation. Each flow is divided with the 5-tuple index (*i.e.*, {Source IP, Source Port, Destination IP, Destination Port, Protocol}) from the network traffic. For each flow,

- I denotes the input, *i.e.*, a packet sequence sent by the client,
- O denotes the output, *i.e.*, a packet sequence received by the server,
- n is the input flow length, *i.e.*, the number of input packets,
- P_i is the i -th input packet, where $i \in [1, n]$,
- m is the output flow length, *i.e.*, the number of output packets,
- R_j is the j -th output packet, where $j \in [1, m]$,
- $M(p_{1...z})$ is the server state after receiving packet sequence $p_{1...z}$, where server state as the output to the buffer that stores application layer data. For instance, the server state can be understood as the corresponding payload, and it could also be the UDP (connectionless protocol) data.

Since TCP is more complicated than UDP, which has mechanisms such as retransmission and rearrangement, the descriptions of network-induced phenomena based on TCP contain that based on UDP, which can be defined as follows.

Definition 1: Synchronized. Given a sequence of packets $P_{1...i} \in I$, and the output packet sequence $R_{1...j} \in O$. We say that $P_{1...i}$ is *synchronized* with $R_{1...j}$ if and only if server state $M(P_{1...i}) = M(R_{1...j})$.

Definition 2: Loss. Given a sequence of packets $P_{1...i+1} \in I$, and the output packet sequence $R_{1...j,j+1...m} \in O$. When $P_{1...i}$ is synchronized (Definition 1) with $R_{1...j}$, we say that the packet P_{i+1} is *lost* if $P_{i+1} \notin R_{j+1...m}$. In other words, P_{i+1} neither causes server state change nor generates any output.

Definition 3: Retransmit. Given a sequence of packets $P_{1...i,i+1...i+x} \in I$ where $x \in [2, n-i]$, and the output packet sequence $R_{1...j} \in O$. When $P_{1...i}$ is synchronized (Definition 1) with $R_{1...j}$, we say that the packet P_{i+x} is the *retransmission* packet of P_{i+1} if $P_{i+1} = P_{i+x}$. In other words, $M(R_{1...j}, P_{i+1}) = M(R_{1...j}, P_{i+x})$.

Definition 4: Out-of-order. Given a sequence of packets $P_{1...i,i+1...i+x} \in I$, and the output packet sequence $R_{1...j,j+1...j+x} \in O$ where $x \in [2, \min(n-i, m-j)]$. When $P_{1...i}$ is synchronized (Definition 1) with $R_{1...j}$, we say there is *out-of-order* between the packet sequence $P_{i+1...i+x}$ and $R_{j+1...j+x}$ if $M(R_{1...j}, P_{i+1...i+x}) = M(R_{1...j}, R_{j+1...j+x})$ and $P_{i+1...i+x} \neq R_{j+1...j+x}$.

3.3 Concerned Network Scenarios

We elaborate here on the typical network scenario. The network quality-of-service (QoS) is the description or measurement of the overall performance of a network service [36]. QoS represents the ability to provide different priorities to different applications, users, or data flows. And it also guarantees a certain level of performance to a data flow, such as a required bit rate, delay, packet loss, or bit error rates. Therefore, QoS is particularly important for traffic transport, and it directly determines the occurrence of network-induced phenomena.

In the wired network, prior arts [35]–[37] have confirmed that bandwidth and topology structures are the main reasons affecting the network QoS. So we mainly consider two network scenarios in our practical wired-network evaluation: (i) Dynamic bandwidth (§ 5.3), which includes TCP burst that exceeds the bandwidth. (ii) Network topology (§ 5.4), which contains two typical structures *i.e.*, parallel and serial. Moreover, the multi-user preemption may affect the wireless QoS [45]. Therefore, we evaluate the multi-user concurrency scenario (§ 6.2) in the wireless experiments.

4 DESIGN DETAILS OF ERNN

4.1 Overview

We first introduce the overview of ERNN design, including feature extraction and recurrent neural network (RNN) unit. We choose long-short term memory (LSTM) as the basic RNN model. Subsequently, we will detail the design of the proposed session gate.

4.1.1 Feature Extraction

Prior arts [10]–[13], [46] have demonstrated that the sequence of packet length is a valid feature as the fingerprint of flow. Therefore, we extract the length from the raw packet

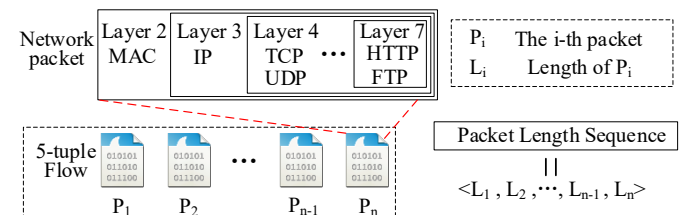


Fig. 2. Feature vector extraction.

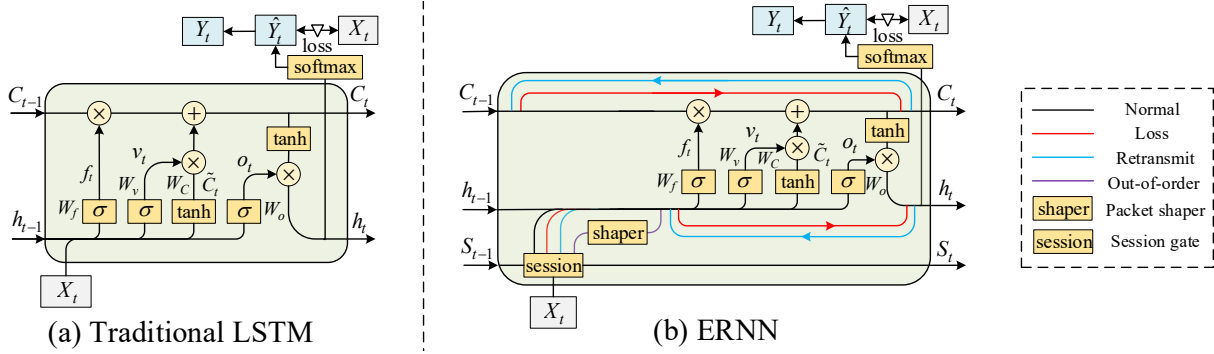


Fig. 3. Traditional LSTM architecture and ours.

sequence as the feature vector of a 5-tuple flow. As shown in Fig. 2, the origins of those lengths range from the data link layer (e.g., MAC) to the application layer (e.g., HTTP). For each flow, we select the first n packets and generate the feature vector as the input of ERNN. If the number of packets is less than n in an individual flow, several zeroes will be padded at the end.

4.1.2 LSTM Unit

Taking long-short term memory (LSTM, *i.e.*, an RNN model) as an example, Fig. 3(a) shows the typical model architecture. In the t -th time step operation of the LSTM unit, the forget gate can be calculated as

$$f_t = \sigma(W_f \cdot [h_{t-1}, X_t] + b_f) \quad (1)$$

where h_{t-1} is the current hidden state and X_t denotes the t -th input. The memory gate v_t can be computed as follows,

$$v_t = \sigma(W_v \cdot [h_{t-1}, X_t] + b_v) \quad (2)$$

And the temporary memory cell $\tilde{C}_t$ is computed by

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, X_t] + b_c) \quad (3)$$

Then, the next cell state can be updated as follows,

$$C_t = f_t \cdot C_{t-1} + v_t \cdot \tilde{C}_t \quad (4)$$

Finally, the output gate o_t can be obtained as

$$o_t = \sigma(W_o \cdot [h_{t-1}, X_t] + b_o) \quad (5)$$

And the next hidden state can be calculated as shown in Eq. (6),

$$h_t = o_t \cdot \tanh(C_t) \quad (6)$$

Particularly, all of the above W and b are learnable parameters.

As mentioned in § 3, packet loss, retransmission, and out-of-order are common network-induced phenomena that exist. However, current methods don't fully consider the influence of these phenomena. Therefore, to realize the robust model against the network-induced phenomena in an error-resilient manner, we introduce **session gate** into traditional RNN during the training process. As shown in Fig. 3(b), we deploy **session gate** after the input X_t . Note that ERNN is not equivalent to preprocessing the input data offline (such as input transformation). Since **session gate** is designed with various actions and state transitions, it can simulate the

randomly generated network-induced phenomenon. Next, we will introduce the design of **session gate** (in § 4.2) and its state update (in § 4.3) in detail.

4.2 Session Gate Design

To construct the traffic session state in the RNN unit, we architect **session gate** as a novel gating unit to sit in ERNN. As shown in Fig. 3(b), **session gate** includes four actions: *pass*; *ignore*; *repeat*; *shape*, and they correspond to the four states, *i.e.*, normal, packet loss, retransmission, out-of-order respectively.

Action 1: Pass. The “pass” action corresponds to the normal flow state, *i.e.*, black lines in Fig. 3(b). In this situation, the process of ERNN is identical to traditional RNN.

Action 2: Ignore. The “ignore” action corresponds to the packet loss state, *i.e.*, red lines in Fig. 3(b). In this situation, ERNN will ignore the input of the current time step and directly transfers the cell and hidden state to the next time step, *i.e.*,

$$\begin{cases} C_t = C_{t-1} \\ h_t = h_{t-1} \end{cases} \quad (7)$$

Eq. (7) means that packet length X_t is ignored and does not change the output of the RNN.

Action 3: Repeat. The “repeat” action corresponds to the retransmission state, *i.e.*, blue lines in Fig. 3(b). In this situation, ERNN will repeat the calculation process of the current time step for once, as shown in Eq. (8)

$$\begin{cases} C_t = U_C(U_C(C_{t-1}, X_t), X_t) \\ h_t = U_h(U_h(h_{t-1}, X_t), X_t) \end{cases} \quad (8)$$

where X_t denotes the input in the t -th time step, U_C and U_h respectively represent a calculation process of cell and hidden state in RNN unit. After the above computation, we can simulate the traffic retransmission and use this packet in the model one more time.

Action 4: Shape. The “shape” action corresponds to the out-of-order state, *i.e.*, purple lines in Fig. 3(b). In this situation, ERNN will introduce the random noise to change the attribute value of some packets within the session, thereby reducing the dependence of the model on the sequential combination of packets, *i.e.*, “shaper” in Fig. 3(b). Specifically, taking the packet length sequence as an example, it selects a random value γ (e.g., $\gamma \in \{0, 8, \dots, 248\}^1$) to modify the

1. This $\{0, 8, \dots, 248\}$ references previous technologies [47] for web fingerprint obfuscation that consider the underlying encryption block size is generally 8 bytes.

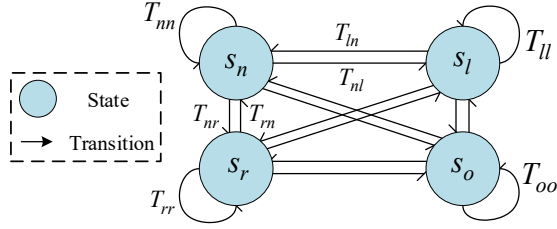


Fig. 4. States and transition of session gate. s_n, s_l, s_r, s_o represent the normal state, loss, retransmission, and out-of-order respectively.

instance X_t , in which X_t denotes the input of the t -th time step. After modification, $X'_t = \min(X_t + \gamma, \text{MTU})$, where MTU means the maximum transmission unit. Intuitively, the packet out-of-order can be approximated as a combination of multiple random noise processes, that is, the confusion of the original sequences. Particularly, other operations, such as reducing the feature value, could be attempted to induce noise in the future. Overall, the original intention of this action is to improve the model generalization ability to cope with the out-of-order phenomenon.

4.3 Session State Update

The current session state S_t determines which action of § 4.2 will be executed. In this section, we focus on how to update the state, which corresponds from S_{t-1} to S_t in Fig. 3(b). Inspired by TCP implementation, we model state transitions of the session gate as a deterministic Mealy machine, $Q = (s_{all}, q_0, \beta, T)$ where

- s_{all} is the set of states.
- $q_0 \in s_{all}$ is the initial state.
- β is the input alphabet. Specifically, it refers to the randomly generated value in each time step and $\beta \in [0, 1]$.
- $T_{xx} : s_{all} \times \beta \rightarrow s_{all}$ is the state transition function, where $x \in \{n, l, r, o\}$. For instance, T_{nl} represents the state transition from normal to packet loss.

Compared with a traditional deterministic finite state machine, the transition of the Mealy machine is determined by both its current state and the input.

Specifically, the state transition process is shown in Fig. 4. There are four states in total, and each state has four transition directions (including self-update). In the RNN unit, the current state S_t is determined by the previous state S_{t-1} and the current transition value v_t . Particularly, v_t is randomly generated as a 32-bit floating point number at each time step, and $v_t \in [0, 1]$. The state transition is executed according to the transition range of S_{t-1} where v_t falls in. For example, suppose the previous state S_{t-1} is the packet loss s_l , and the probability distributions of s_l is 0.1, 0.2, 0.3, 0.4 for normal (T_{ln}), packet loss (T_{ll}), retransmission (T_{lr}), out-of-order (T_{lo}), respectively. This means that the transition range of s_l is normal = $[0, 0.1]$, loss = $(0.1, 0.3]$, retransmission = $(0.3, 0.6]$, and out-of-order = $(0.6, 1]$. If $v_t = 0.1$, the current state is transitioned to “normal”. And if $v_t = 0.9$, the current state will be “out-of-order”.

In our implementation, we use a 4×4 matrix F_m to store the transition relationship. We provide proposal F_m value ranges for a wired network in online repository². Particularly,

2. <https://github.com/TDSCCode/ERNN/tree/main/transition>.

the value range in the transition matrix can be adjusted according to specified network situations. For instance, we can increase the packet loss rate by 0.1 for a wireless network. When the topological structure is complex, the probability of out-of-order can be increased by 10%. Furthermore, we also evaluate the detection effect of ERNN with different parameter settings in § 7.1.

4.4 Addressing Design Concerns

Use Instructions. The session gate is designed to simulate network-induced phenomena to improve the error-resilient capability. Therefore, we clarify that this gating mechanism is mainly enabled during the training phase and disabled during the testing phase. Specifically, in the testing phase, the probability of the next state being normal is 100%, regardless of what the current state is. Such usage does not further degrade the quality of test data considering there may already be network-induced phenomena.

Quality of the Training Set. In addition, it has to be admitted that the network-induced phenomenon could exist when collecting the training dataset. For this problem, we consider countermeasures from two aspects. On the one hand, maintaining a high network quality as much as possible when capturing the training set, and conducting multiple data collections to mitigate the impact of accidental errors. On the other hand, since the dataset collection environment is a controllable scene, we could check and correct problematic packets by comparing the sender-side and receiver-side logs. This dataset calibration in a controlled collection environment is also beneficial for feasibility in practice. Moreover, § 7.3 shows that ERNN has good elasticity when the training set holds a small ratio of network-induced phenomena.

5 EVALUATION: INTRUSION DETECTION

We fully implemented a prototype of ERNN with C and Python.³ Our evaluations center around demonstrating the error-resilient properties of ERNN against network-induced phenomena.

Parameter Settings. We elaborate here on the parameter settings of ERNN. Specifically, the model is designed as a two-layer architecture. For each layer, it includes 128 time steps, which means the first 128 packets will be analyzed for each flow ($n = 128$). Moreover, the dimension of neurons in the hidden layer is set to 400, the learning rate is 1e-3, and we adopt cross-entropy as the loss function.

We set the initial state q_0 as normal, i.e., $q_0 = s_n$. Meanwhile, we set the transition matrix F_m of session state to be within the proposed range⁴. The details are shown in Eq. (9).

$$F_m = \begin{pmatrix} 0.7 & 0.1 & 0.1 & 0.1 \\ 0.85 & 0.05 & 0.05 & 0.05 \\ 0.85 & 0.05 & 0.05 & 0.05 \\ 0.85 & 0.05 & 0.05 & 0.05 \end{pmatrix} \quad (9)$$

Testbed Setup. To simulate the network-induced phenomena, we realize the network forwarding program based on DPDK-20.11 [48]. The dataplane is multi-threaded where the forwarding thread is built using DPDK's Internet Protocol

3. Available at <https://github.com/TDSCCode/ERNN>.

4. <https://github.com/TDSCCode/ERNN/tree/main/transition>.

Pipeline block, and the meter thread is built based on a tool called *pdump*. Subsequently, the program is compiled and linked with the DPDK libraries specified in the control plane.

Our physical testbed has a 10 Gbps dataplane between two hosts: one serves as a packet generator and the other deploys ERNN. The packet generator is equipped with two 10 GbE Intel X540-AT2 network cards, an Intel i7-9700 CPU (3.00 GHz, 8 cores), and 16 GB memory. The second host, deployed with ERNN, has the same configuration as the first, except only a 10 GbE Intel X540-AT2 network card. Both hosts are running Ubuntu 16.04.1 LTS with Linux kernel 4.15.0, and their bandwidth difference is the key of simulating bandwidth overflow. We use pktgen-dpdk 20.03.0 for traffic generation. Multiple 10 GbE HUAWEI S6720-32C-SI-AC switches (24 ports) are used to connect two hosts and build various topology structures mentioned in § 5.4.

Particularly, we use the *scapy* library to modify the corresponding IP and MAC addresses in the PCAP file to implement the replay process. Our switches are based on MAC addressing that can transmit packets according to the destination MAC address field. Note that DPDK supports the receive side scaling (RSS) which can guarantee per-connection consistency (PCC) [49], *i.e.*, same-flow packets are processed in one core when multi-core parallel. This means that multi-core parallel does not cause additional packet out-of-orders etc.

TABLE 2
Datasets used in our evaluation of intrusion detection.

Dataset	Attack Type	Description
IDS2017	FTP-Patator, SSH-Patator, DoS-GoldenEye, DoS-Slowloris, DoS-SlowHTTPTest, DoS-Hulk, DDoS, Web-Attack, Infiltration, Botnet, Heartbleed, Port-Scan	12 common attacks+benign
IDS2018	FTP-Patator, SSH-Patator, DoS-GoldenEye, DoS-Slowloris, DoS-SlowHTTPTest, DoS-Hulk, DDoS, Web-Attack, Infiltration, Botnet	10 common attacks+benign

5.1 Dataset and Baseline

Datasets. To facilitate a fair comparison, we use two public datasets IDS2017 [50] and IDS2018 [51] for our evaluations. The IDS2017 consists of 225,745 packets collected from specific network activities, including both normal and intrusion activities, for more than seven days. The IDS2018 contains 16,233,002 instances based on the network traffic of ten days. Both datasets use a proposed B-Profile system [50] to profile the abstract behavior of human interactions⁵. Moreover, we use the traffic generated in normal scenarios as the benign ones. As shown in Table 2, the datasets can be divided into six categories, containing benign and 5 attacks. In this experiment, we divide all PCAP traffic into 5-tuples flow and extract the feature vector (such as packet length sequence). We conduct multi-classification tasks in intrusion detection evaluation, *i.e.*, identifying specific types of attacks.

Metrics. The main metrics for performance evaluation are overall *Acc* and *F1*, as detailed in our online repository⁶.

Baselines. For comparison, some state-of-the-art methods are briefly introduced as follows:

- FoSM [9] uses message type sequences to build a first-order Markov model and takes the application with the maximum probability as the classification result.
- FoLM is a variant of FoSM, which replaces the message type sequences with the packet length sequences.
- SOCRT [11] classifies applications combined with the certificate packet length and the second-order Markov model. The certificate clustering number is set as 5.
- SOB [12] imports the first communication packet length based on SOCRT to decide the final classification results. We take 40 as the number of bi-gram clustering.
- MaMPF [10] uses the output probabilities of the message type and the packet length Markov models as features, to classify encrypted traffic with the random forest (named as SLC-RF) or logistic regression (named as SLC-LR). The number of trees is set as 50.
- FS-Net [13] uses the bi-GRU model to learn the sequence features and classify common applications.
- Normal LSTM is also considered since our design (in § 4) is based on the traditional LSTM.

5.2 Evaluation on Robustness

To evaluate the influence of network-induced phenomena on ERNN and baseline models, we randomly generate test data including normal, packet loss, retransmission, and out-of-order (with controllable probability). The specific generation process leverages the *scapy* library to manipulate the packets from PCAP files. Given respective corresponding probabilities of mimics network-induced phenomena, among them, “loss” means removing the packet, “retransmission” refers to copying the packet, and “out-of-order” represents swapping packets of two different positions. Note that these operations are performed inner-session. The results are shown in Table 3. Among them, the first row with ID_0 shows the prediction accuracy and F1 score without network-induced phenomena. Moreover, the $ID_1 \sim ID_9$ display the situations with only one network-induced phenomenon. The $ID_{10} \sim ID_{18}$ and $ID_{19} \sim ID_{21}$ have two and three phenomena, respectively. We use R_l , R_r , R_o to denote the probability of packet loss, retransmission, and out-of-order, where $R_l, R_r, R_o \in \{0.05, 0.1, 0.2\}$.

Comparing the results of $ID_1 \sim ID_9$, we find that the packet loss introduces the most significant influence on RNN-based models, *i.e.*, the accuracy of FS-Net, LSTM, and ERNN are 81.9%, 51.5%, and 90.8% when $R_l = 0.2$ (ID_3). While out-of-order is the most dominant phenomenon that causes the Markov-based model misclassification, *e.g.*, the accuracy of FoLM, SOB, and SLC-RF are 36.5%, 44.7%, and 67.7% respectively when $R_o = 0.2$ (ID_9).

For $ID_{10} \sim ID_{18}$ which have two network-induced phenomena, the combination of packet loss and out-of-order presents the greatest impact on all models. Specifically, the accuracy of FoSM, FoLM, SOCRT, SOB, SLC-LR, SLC-RF, FS-Net, LSTM, and ERNN are 29.9%, 31.6%, 33.8%, 38.9%, 53%, 56.7%, 81.8%, 50.3%, and 90.5% respectively when $R_l = R_o = 0.2$ (ID_{15}). In $ID_{10} \sim ID_{18}$, an interesting observation is that the combination of packet loss and retransmission has the least influence on all models, *e.g.*, the accuracy of FS-Net, LSTM, and ERNN are 98%, 96.6%, and 99.2% respectively when $R_l = R_r = 0.05$ (ID_{10}). The reason could be explained

5. Including multiple protocols (TCP, UDP, ICMP, etc.) and multiple applications (FTP, HTTP, DNS, NetBIOS, SNMP, SSH, Messenger, etc.).
6. <https://github.com/TDSCCode/ERNN/tree/main/metrics>.

TABLE 3

Model robustness experiment against the diverse network-induced phenomenon. Particularly, R_l represent the ratio of packet loss; R_r represent the ratio of retransmission; R_o represent the ratio of out-of-order. Acc (%) and F1 (%) represent accuracy and F1-score respectively.

ID	Phenomenon				FoSM		FoLM		SOCRT		SOB		SLC-LR		SLC-RF		FS-Net		LSTM		ERNN	
	R_l	R_r	R_o		Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
0	0	0	0		73.6	72.1	79.2	78.4	82.6	81.6	90.5	90.3	91.3	91.2	95.4	94.6	99.0	98.6	99.3	99.1	99.2	99.2
1	0.05	0	0		61.3	62.0	66.3	65.9	69.6	69.5	71.9	71.3	84.0	83.8	91.9	91.3	93.1	93.0	91.0	90.0	96.3	95.9
2	0.1	0	0		47.9	46.5	50.1	49.2	57.6	60.1	63.4	62.0	76.2	75.4	84.7	85.1	89.5	88.6	85.3	84.5	93.0	92.0
3	0.2	0	0		39.0	36.7	41.3	42.1	44.8	48.0	54.3	55.1	69.4	68.1	72.2	73.5	81.9	81.4	51.5	49.7	90.8	90.2
4	0	0.05	0		65.5	66.0	70.1	69.9	71.4	70.8	75.8	74.1	85.3	85.0	92.8	92.0	97.8	97.6	96.9	96.9	99.2	99.2
5	0	0.1	0		53.4	52.8	58.3	57.7	62.4	63.8	68.1	62.9	80.6	79.8	88.3	88.3	95.2	94.9	93.0	93.1	99.2	99.2
6	0	0.2	0		43.9	44.2	49.4	51.0	53.4	56.0	57.9	56.6	74.4	72.7	81.3	82.1	86.7	86.3	70.1	67.8	96.4	96.3
7	0	0	0.05		56.9	56.3	61.0	60.7	64.5	64.3	67.8	63.9	78.7	77.6	90.7	90.3	98.5	98.4	97.3	97.3	99.2	99.2
8	0	0	0.1		41.8	44.7	45.4	46.6	48.9	51.5	54.6	55.3	65.2	63.2	79.4	80.4	97.3	97.2	96.5	96.5	99.1	99.1
9	0	0	0.2		32.2	30.3	36.5	34.9	38.4	40.8	44.7	45.6	55.5	53.2	67.7	69.2	96.5	96.8	95.4	95.4	98.8	98.8
10	0.05	0.05	0		58.7	57.9	61.2	60.8	67.1	67.2	71.0	70.6	82.1	81.5	91.6	91.1	98.0	97.8	96.6	96.6	99.2	99.2
11	0.1	0.1	0		45.9	47.3	49.2	50.3	52.3	55.1	58.3	57.7	71.8	70.4	82.4	83.1	94.6	94.2	92.3	92.3	98.9	98.9
12	0.2	0.2	0		34.6	35.7	36.8	37.3	39.2	41.8	46.9	47.3	60.1	58.2	67.5	67.5	89.2	89.0	77.0	76.0	92.4	91.3
13	0.05	0	0.05		53.6	53.3	58.1	58.3	62.8	62.9	68.5	68.2	77.4	76.2	89.7	89.4	92.9	92.4	90.9	89.9	93.7	92.7
14	0.1	0	0.1		39.7	40.3	43.9	44.1	46.2	49.1	51.6	51.9	64.4	62.6	75.1	76.5	87.7	87.0	84.5	83.6	92.9	91.9
15	0.2	0	0.2		29.9	30.7	31.6	32.4	33.8	35.1	38.9	37.7	53.0	49.8	56.7	57.5	81.8	81.1	50.3	49.2	90.5	90.1
16	0	0.05	0.05		54.6	53.9	57.0	58.1	63.9	63.9	69.1	68.5	78.4	77.1	90.6	90.1	97.9	97.7	96.8	96.8	99.2	99.2
17	0	0.1	0.1		38.1	37.7	41.9	42.4	46.7	49.5	52.6	53.1	65.7	63.6	76.4	77.5	93.9	93.8	91.4	91.5	99.0	99.0
18	0	0.2	0.2		31.7	30.9	34.2	36.5	36.6	38.8	41.9	41.6	54.8	52.5	64.3	64.7	83.2	83.5	68.6	66.1	95.0	94.9
19	0.05	0.05	0.05		52.1	52.6	57.6	57.9	62.2	62.2	67.4	68.0	77.0	75.5	89.9	89.5	97.6	97.4	96.5	96.5	99.2	99.2
20	0.1	0.1	0.1		39.5	39.7	42.2	41.6	44.9	47.7	49.6	50.1	63.4	61.4	74.3	75.2	94.0	93.7	91.6	91.6	98.8	98.8
21	0.2	0.2	0.2		26.0	24.4	29.5	29.2	32.9	34.1	37.4	38.1	49.2	46.2	57.9	58.1	86.3	85.1	76.0	75.0	91.4	90.4

as the loss of the retransmitted packets. As for $ID_{19} \sim ID_{21}$, it brings the most misclassification on Markov-based models, *i.e.*, the accuracy of FoSM, FoLM, SOCRT, SOB, SLC-LR, and SLC-RF are 26%, 29.5%, 32.9%, 37.4%, 49.2%, and 57.9% respectively when $R_l = R_r = R_o = 0.2$ (ID_{21}). In comparison, the accuracy of FS-Net, LSTM, and ERNN remain 86.3%, 76%, and 91.4%.

Overall, ERNN performs the best classification result with accuracy and F1-score greater than 90% in all scenes. Comparing ERNN and normal LSTM, we can find that the design of session gate enhances the model robustness in an error-resilient manner. Particularly, the design of session gate in ERNN does not require specific constraints, and it is generic for the traditional RNN model. This means that once the new RNN model is realized, its capabilities can also benefit from ERNN's error-resilient design.

5.3 Evaluation on Dynamic Bandwidth Utilization

Some literatures [35], [36] demonstrated that network quality-of-service (QoS) will be affected by bandwidth. Therefore, we focus on evaluating the capabilities of ERNN against dynamic bandwidths in this section. Toward this end, we generate traffic with diverse bandwidths (0 ~ 15 Gbps) on the dataplane. Particularly, we select 7 stages with bandwidths 2G (20%), 4G (40%), 6G (60%), 8G (80%), 10G (100%), 12G (120%), and 14G (140%), and statistic the corresponding network-induced

phenomena. The ratio of packet loss, retransmission, and out-of-order is shown in Table 4. Among them, the out-of-order ratio includes two levels: the packet level and the flow level.

(i) For the packet level, we put all packets in one set and analyze it directly. The "Num" represents the proportion of out-of-order packets on all packets. The "Degree" represents the proportion of *inversion number* to the total packets number. For instance, consider a send packet sequence $\{1, 2, \dots, n-1, n\}$, where n denote the number of packets. And the receive packet sequence is $\{rec_1, rec_2, \dots, rec_{n-1}, rec_n\}$. The *inversion number* N_{inv} is equal to the number that satisfies $rec_i < rec_j$ and $i > j$, where $i, j \in [1, n]$. Then "Degree" can be calculated as $\frac{2 \times N_{inv}}{n \times (n-1)}$.

(ii) For flow level, we divide all packets according to the 5-tuple index and analyze each flow separately. The "Num" represents the proportion of out-of-order packets (after summing every flow) on all packets. To compute "Degree", we need to record the number of packets and the *inversion number* of each flow. Specifically, for a flow set $\{flow_1, flow_2, \dots, flow_{n-1}, flow_n\}$, supposing the number of packets is $\{num_1, num_2, \dots, num_{n-1}, num_n\}$ and the *inversion number* is $\{inv_1, inv_2, \dots, inv_{n-1}, inv_n\}$. The "Degree" can be computed as $\frac{2 \times (inv_1 + inv_2 + \dots + inv_{n-1} + inv_n)}{num_1 \times (num_1 - 1) + \dots + num_n \times (num_n - 1)}$.

From Table 4, we find that when the bandwidth utilization is less than 60%, the network-induced phenomenon is mainly out-of-order. Meanwhile, the retransmission phenomenon gradually increases when the bandwidth utilization is from 60% to 100%. Once the bandwidth is higher than 100% (such as TCP burst), the packet loss rate increases significantly due to the buffer overflow.

The model accuracy for different dynamic bandwidth utilization can be depicted in Fig. 5. Specifically, Fig. 5(a) on the top displays how the throughputs (denoted R_p in Gbps) vary with respect to time t at the sender side, where the packet sending rate reaches to 2 Gbps, 4 Gbps, 6 Gbps, 8 Gbps, 10 Gbps, 12 Gbps, and 14 Gbps at $t = 2, 7, 12, 17, 22, 27, 32$, respectively. And each stage (*e.g.*, 2 Gbps, 4 Gbps)

TABLE 4

Network-induced phenomena in different bandwidth utilization.

Rate R_p (GB)	Loss R_l (%)	Retransmit R_r (%)	Out-of-order R_o (%)			
			Packet		Flow	
			Num	Degree	Num	Degree
2	0.004	0.482	3.433	1.517	2.086	0.523
4	0.005	0.837	3.889	1.897	2.528	0.638
6	0.005	2.763	5.648	2.538	3.223	0.867
8	0.008	4.439	7.565	4.072	5.638	1.841
10	0.018	7.203	9.300	6.611	8.805	2.997
12	12.243	20.802	28.315	13.669	25.116	10.750
14	19.867	26.283	39.502	22.523	34.967	16.751

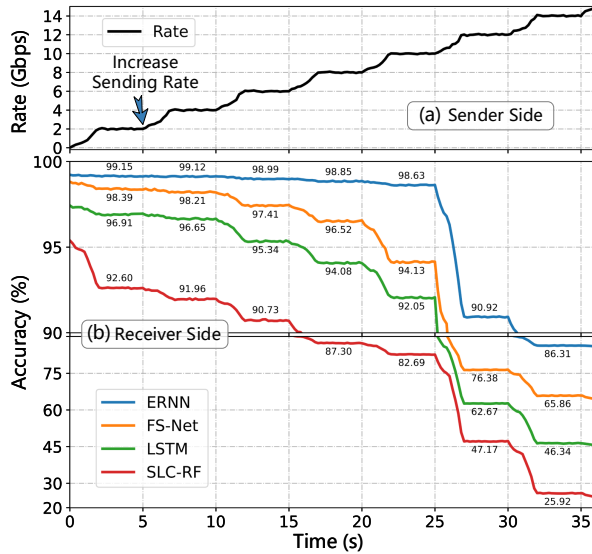


Fig. 5. Model accuracy in dynamic bandwidth.

lasts for stable bandwidth of 3 seconds. For $t \in [0, 15]$, R_p is only about 0 ~ 6 Gbps as the bandwidth is sufficient to support fine quality-of-service (QoS). For $t \in [15, 25]$, R_p further increases and eventually reaches the dataplane upper limit (10 Gbps). For $t \in [25, 35]$, we set up two stable stages (i.e., 12 Gbps and 14 Gbps) to simulate TCP burst and distributed denial of service attack (DDoS).

Fig. 5(b) at the bottom reveals how the model classification accuracy at the receiver side changes in corresponding to the dynamic bandwidths. When $t \in [0, 15]$, ERNN is only suffered negligible influence, and its accuracy is greater than 98.99%. While the accuracy of the other three models (FS-Net, LSTM, and SLC-RF) has been suffered a certain drop, from 98.39%, 96.91%, and 92.6% down to 97.41%, 95.34%, and 90.73% respectively, at $t = 15$. For $t \in [15, 25]$ where R_p is greater than 60% bandwidth, all models introduce some misclassification due to the network congestion. Specifically, the accuracy of ERNN, FS-Net, LSTM, and SLC-RF is 98.63%, 94.13%, 92.05%, and 82.69% respectively at $t = 25$. For $t \in [25, 35]$ where the bandwidth exceeds 10 Gbps, the model accuracy decreases significantly due to lots of packet loss introduced by the buffer overflow. Nonetheless, ERNN can still correctly classify more than 85% flows at $R_p = 14$ Gbps, while the accuracy of other models is less than 70%. In summary, compared to these state-of-the-art (SOTA) methods, ERNN presents the best classification accuracy against dynamic bandwidth utilization. Particularly, the classification accuracy of ERNN is about 20% higher than other models at $t = 35$ ($R_p = 14$ Gbps).

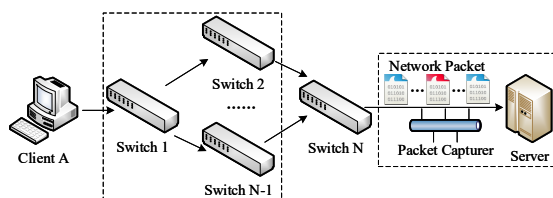


Fig. 6. Network topology with parallel structure.

TABLE 5
Network-induced phenomena in parallel topology structure.

Switch N	Loss R_l (%)	Retransmit R_r (%)	Out-of-order R_o (%)			
			Packet		Flow	
			Num	Degree	Num	Degree
3	0.003	1.465	2.412	1.042	1.853	0.327
4	0.005	1.848	4.715	2.256	2.944	0.756
5	0.006	2.019	8.368	5.485	6.744	2.486
6	0.008	2.668	10.182	6.781	8.934	4.210
7	0.009	3.416	11.969	8.665	10.114	4.973

5.4 Evaluation on Different Network Topology

To further demonstrate its effectiveness, we evaluate ERNN in two typical network topologies (mentioned in § 3.3), addressing complicated network scenarios.

5.4.1 Case 1: Parallel Structure

First, we change the number of switch links between the two hosts and construct a parallel architecture. Fig. 6 shows the parallel network topology structure, where N denoted the number of network switches. And the number of network hops is 4. Note that the switches support the spanning tree protocol (STP) to prevent network loops. Moreover, we generate traffic with 50% bandwidth on the dataplane.

Table 5 shows the statistical results of network-induced phenomena. The calculations of packet loss, retransmission, and out-of-order are same as § 5.3. With more switches used (N increases from 3 to 7), the packet loss rate changes slightly, i.e., R_l from 0.003% to 0.009%. While the probability of out-of-order has increased significantly, e.g., R_o increases by more than 10% for both packet and flow level at $N = 7$. Moreover, the retransmission probability gradually rises (R_r from 1.465% to 3.416%).

We illustrate the model classification accuracy in Fig. 7. It can be seen that the impact of increasing parallel switch links on ERNN is negligible, where its accuracy is 98.87% at $N = 7$. As for the two RNN-based baseline models, they have similar descending curves, where the accuracy of FS-Net and LSTM is 94.44% and 93.46% when $N = 7$. However, the accuracy curve of SLC-RF (based on Markov) falls to 81.62% very quickly.

5.4.2 Case 2: Serial Structure

Second, we evaluate the capability of ERNN against serial network structure. Specifically, Fig. 8 shows the serial network topology structure, where N denotes the number of network switches. And the number of networks hops is

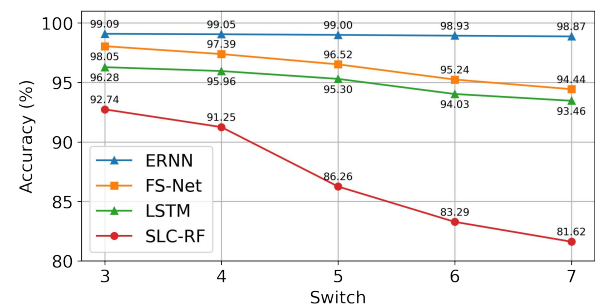


Fig. 7. Model accuracy in parallel network structure.

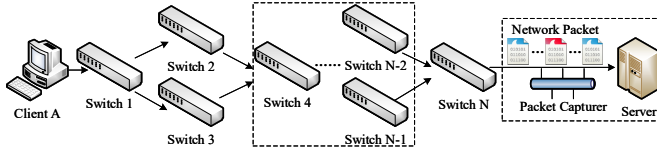


Fig. 8. Network topology with serial structure.

$(2 \times N + 1)/3 + 1$. Similarly, the spanning tree protocol (STP) will be used to prevent network loops. And the speed of packet generation is 50% bandwidth.

TABLE 6
Network-induced phenomena in serial topology structure.

Switch N	Hop	Loss R_l (%)	Retransmit R_r (%)	Out-of-order R_o (%)			
				Packet		Flow	
				Num	Degree	Num	Degree
4	1	0.005	1.854	4.698	2.263	2.936	0.759
7	2	0.007	3.242	9.901	6.899	6.831	2.635

The statistical results of packet loss, retransmission, and out-of-order are displayed in Table 6. It is obvious that packet out-of-order is still the most frequent network-induced phenomenon. We demonstrate the model classification accuracy in Fig. 9. With the increase of a group of serial switches (*e.g.*, the number of hops is increased by 2), it will only bring about 0.1% misclassification on ERNN. Furthermore, the extra serial switches will cause an accuracy drop of about 2% for FS-Net and LSTM, and 5% for SLC-RF, respectively.

Overall, ERNN can maintain high accuracy regardless of whether serial or parallel switch links are introduced to the network topology. In our evaluation, the accuracy drop of ERNN caused by topology structure change is less than 0.5%, which shows that ERNN has good error-resilient capability against network-induced phenomena in practical scenarios.

6 EVALUATION: MALWARE DETECTION

To further analyze the more AI-based cybersecurity solutions, we evaluate here ERNN in malware detection.

Testbed Setup. In the landscape of malware detection, dynamic features-based approaches [52], [53] aim to identify the malware with the traffic generated during operations. We consider a typical local area network scenario and our testbed as shown in Fig. 10. Specifically, it displays a multi-user concurrency scenario that exists X devices are connected to the local-area wireless. Among them, device 1 runs malware, and other devices run benign Apps. The generated traffic of all devices is mirrored from the router to the malware detection equipment which executes the model inference.

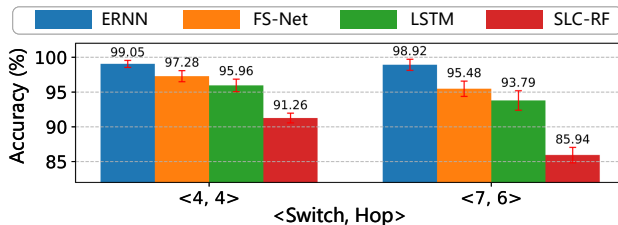


Fig. 9. Model accuracy in serial network structure.

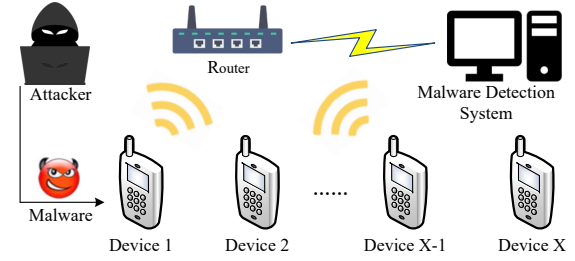


Fig. 10. Malware detection in multi-user concurrency scenario.

In our physical testbed, the router is TP-LINK TL-XVR6000L and the malware detection equipment installs an Intel i7-9700 CPU (3.00 GHz, 8 cores), a GTX 1070 GPU, and 16 GB memory. Every device is running Ubuntu 16.04.1 LTS with Linux kernel 4.15.0, installs Android SDK(30.0.5), Androguard(3.5), and Tcpdump(4.99.1), and configures the official Android emulator [54] to run applications with a graphical user interface.

6.1 Dataset and Baseline

Dataset. In this experiment, the public dataset CICAndMal2017 [55] is used to evaluate the model performance. As shown in Table 7, 4,354 malware samples in the CICAndMal2017 are almost evenly classified into four categories: Adware, Ransomware, Scareware, and SMS Malware. Among them, each category contains many families (*e.g.*, Adware includes Dowgin, Ewind, Feiwo, etc.). Moreover, 6,500 instances of benign Apps are collected from Googleplay market published in 2015, 2016, 2017. We conduct binary classification in malware detection evaluation given that most baselines are binary classification models such as Kitsune and Whisper.

Baselines. We select some state-of-the-art methods from malware detection are summarized as follows:

- Kitsune [15] detects abnormal behavior by using AutoEncoder on each packet instead of flow information.
- TFSVM [52] constructs the one-class support vector machine (SVM) classifiers by using a weighted linear Kernel trained on the tree-based features that can identify malware based on the traffic of Apps.
- OIRTraffic [53] obtains statistical feature representation (*i.e.*, feature histogram) computed from the network traffic and leverages SVM to recognize malicious behavior.
- Whisper [44] utilizes sequential information based on the frequency domain features to detect malicious traffic.

TABLE 7
Datasets used in our evaluation of malware detection.

Category	Family
Benign	6,500 benign Apps from Googleplay market
Adware	Dowgin, Ewind, Feiwo, Gooligan, Kemoge, Koodous, Mobidash, Selfmite, Shuanet, Youmi
Ransomware	Charger, Jisut, Koler, LockerPin, Simplocker, Pletor, PornDroid, RansomBO, Svpeng, WannaLocker
Scareware	AndroidDefender, AndroidSpy, AV for Android, AVpass, FakeApp, FakeApp.AL, FakeAV, FakeJobOffer, FakeTaoBao, Penetho, VirusShield
SMS Malware	BeanBot, Biige, FakeInst, FakeMart, FakeNotify, Jifake, Mazarbot, Nandrobox, Plankton, SMSsniffer, Zsone

TABLE 8
Network-induced phenomena in multi-user concurrency scenario.

Device X	Loss R_l (%)	Retransmit R_r (%)	Out-of-order R_o (%)			
			Packet		Flow	
			Num	Degree	Num	Degree
5	0.166	2.355	3.272	1.472	2.184	0.613
10	0.251	4.557	4.637	2.139	3.398	0.925
15	0.294	6.778	5.687	2.538	4.239	1.316
20	0.337	7.823	6.595	3.724	5.016	1.741
25	0.358	8.905	7.447	4.013	5.945	2.206

- Normal LSTM is also considered since our design (in § 4) is based on the traditional LSTM.

6.2 Evaluation on Multi-User Concurrency

In Fig. 10, we set $X = 5, 10, 15, 20, 25$ respectively and calculate the network-induced phenomena. As Table 8 shows, the packet loss rate in wireless (0.166%~0.358% R_l) is significantly higher than the wired network (~0.01% R_l). With more concurrent devices, the retransmission rate increases rapidly while the out-of-order rate changes slightly, *i.e.*, R_r from 2.355% to 8.905% and R_o from 0.613% to 2.206% in the flow level.

In Table 9, we test ERNN and five baseline models, in the offline dataset (Column1) and five groups of concurrent experiments (Column2~6 correspond to the device number $X = 5, 10, 15, 20, 25$). In the offline datasets with hardly network-induced phenomena, the performances of several models are evenly matched, *e.g.*, the accuracy of ERNN, Kitsune, Whisper, and LSTM is 99.1%, 99.0%, 99.3%, and 99.2%, respectively. However, in the real multi-user concurrency scenario, their performances are affected to varying degrees. Specifically, when $X = 25$, the overall detection performance is Whisper > Kitsune > OIRTraffic > TFSVM (from ~93% to ~91% accuracy), while the traditional LSTM drops to ~89% Acc. Nonetheless, ERNN can still realize over 97% accuracy, which is ~4% more than SOTA methods. Particularly, as a recent anomaly-based SOTA model, Whisper proposes utilizing frequency domain features against adversaries that inject benign traffic into the malicious traffic. Comparing ERNN with Whisper, we observe that the accuracy of Whisper is slightly higher than ERNN (~0.2%) when there is no network-induced phenomenon, while the error-resilient capability of ERNN outperforms Whisper. For instance, when $X = 25$, ERNN presents 1.9% accuracy loss, yet Whisper's ACC decreases by 5.8%, which is ~3 times that of ERNN.

To explore the robustness of Whisper against network-induced phenomena, we randomly select 10K instances and

TABLE 9
Model performance in multi-user concurrency scenario.

Device X		offline	5	10	15	20	25
Kitsune	Acc	99.0	97.2	96.0	94.9	93.8	92.9
	F1	98.9	97.2	95.9	95.0	93.7	92.7
TFSVM	Acc	98.4	96.8	95.1	93.6	92.4	91.4
	F1	98.4	96.5	94.9	93.6	92.1	91.3
OIRTraffic	Acc	98.8	96.9	95.7	94.5	93.3	92.4
	F1	98.7	96.8	95.8	94.3	93.2	92.1
Whisper	Acc	99.3	97.6	96.5	95.3	94.6	93.5
	F1	99.3	97.5	96.5	95.4	94.5	93.3
LSTM	Acc	99.2	96.1	93.9	92.0	90.9	89.9
	F1	99.1	96.0	93.7	91.8	90.5	89.6
ERNN	Acc	99.1	98.4	98.0	97.6	97.5	97.2
	F1	99.1	98.2	97.9	97.4	97.3	97.0

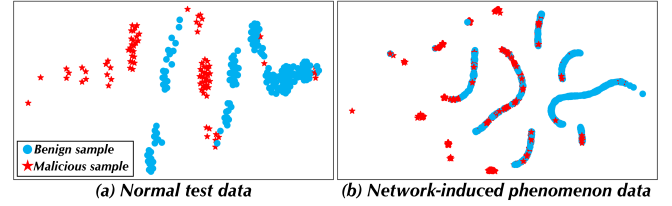


Fig. 11. Visualizing frequency domain features extracted by Whisper.

use t-SNE [56] to visualize the frequency domain features generated by Whisper on the normal test set and problematic test set ($R_l = 0.2$; $R_r = 0.2$; $R_o = 0.2$) respectively. By comparing subfigures 11 (a) and (b), it is clear that the network-induced phenomenon causes Whisper's frequency domain features to lose great separability between benign and attack samples. We speculate that this is caused by Whisper focusing on the adversaries that inject benign traffic into the malicious traffic, which differs from the network-induced phenomena. For example, packet loss may cause some packets to be lost directly, while out-of-order may cause malicious packets to swap positions. These scenarios would change the frequency domain representation in Whisper, further leading to misclassification.

6.3 Overhead Evaluations

We measure the model inference time and memory utilization for LSTM and ERNN. In Fig. 12(a), the traditional LSTM will need about 123.7 μ s to execute per-sample inference on average. And ERNN uses about 124.5 μ s that only introduces less than 0.1% latency. For memory overhead, ERNN and LSTM differ less than 0.01 MB. Overall, ERNN imposes negligible inference time and memory overhead.

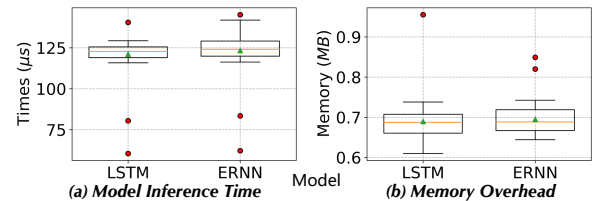


Fig. 12. Inference time and memory overhead.

6.4 Inference Process Profiling

To further analyze the effect of ERNN, we visualize the model inference process in Fig. 13. In all subgraphs, the x-axis represents the time step of the input, each time step refers to the per-packet feature. The y-axis represents the softmax probability corresponding to the current hidden output. The model predicted result is malicious when the softmax probability in the final time-step (*i.e.*, 128) is over 0.5, otherwise benign.

Fig. 13(a) and (b) show the model inference process of traditional LSTM when against network-induced phenomena. Among them, all the test samples in part (a) are benign, and the malicious in part (b). In both Fig. 13(a) and (b), the traditional LSTM presents many misclassification (corresponding to the red line). Since the network-induced phenomenon, the

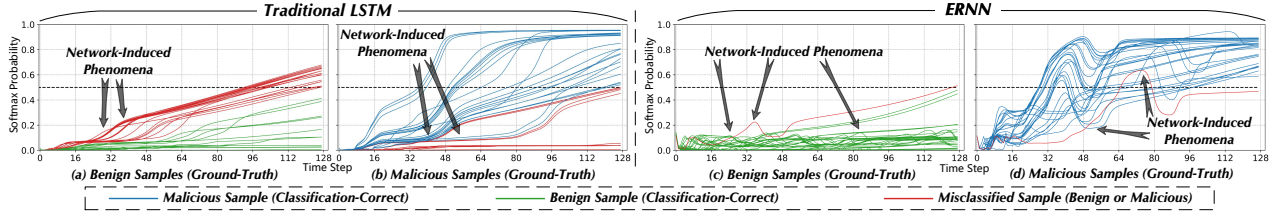


Fig. 13. The visualization of the model classification process in network-induced phenomena.

abnormal packet sequence changes the original prediction curve, such as the turning point of the curve in Fig. 13. However, ERNN appears few misclassifications in both Fig. 13(c) and (d). It is clear that ERNN can quickly recover the curve orientation after suffering from network-induced phenomena to alleviate this negative effect, *e.g.*, local jitter of the prediction curve in subgraphs (c) and (d).

Overall, comparing subgraphs (a)(b) and (c)(d), we can find the traditional LSTM shows a very smooth inference process so that it is easily affected by abnormal packet sequences (network-induced phenomena). Yet ERNN presents local jitter to adversary the negative effect from abnormal packet sequences, thereby achieving the correct prediction process in the global. Therefore, ERNN is robust and reliable to alleviate the model accuracy loss against packet loss, retransmission, and out-of-order in the real world.

7 DEEP INSIGHTS INTO ERNN

In this section, we will further explore the principle behind this error-resilient mechanism in ERNN. To this end, we use the Integrated Gradients method [57] from the Captum [58] to generate the attribution matrix to provide the model interpretability. For one thing, we vary the transition matrix settings to evaluate the elasticity of ERNN, and interpret the role of the gating mechanism, as well as discuss the instructions for real-world deployment. For another, more feature fields will be used for the model inference which expounds on the extensibility of ERNN.

7.1 Transition Matrix

Elasticity of Different Settings. In the experiment, we set two variables ($x, y \in [0.6, 1.0]$) to denote the probability from “normal” to “normal” and “abnormal” to “normal”, respectively. As shown in Eq. (10), T_{nn} is set to x and all T_{nl}, T_{nr}, T_{no} are $(1-x)/3$. For rows 2~4 of the transition matrix, the settings are the same, *i.e.*, $T_{ln} = T_{rn} = T_{on} = y$ and others are $(1-y)/3$.

$$F_m = \begin{pmatrix} x, & (1-x)/3, & (1-x)/3, & (1-x)/3 \\ y, & (1-y)/3, & (1-y)/3, & (1-y)/3 \\ y, & (1-y)/3, & (1-y)/3, & (1-y)/3 \\ y, & (1-y)/3, & (1-y)/3, & (1-y)/3 \end{pmatrix} \quad (10)$$

Particularly, this evaluation is based on the intrusion detection dataset given it is a multi-classification task (should be more difficult than binary). Each set of experiments is performed 10 times with different random seeds (such as `numpy.random.seed` and `torch.manual_seed`).

The experimental results of detection accuracy (with standard deviation) are summarized in the heatmap parts

of Fig. 14. In each heatmap, the first row where $x = 1.0$ refers to typical LSTM⁷, and the other 20 squares present ERNN with different parameter settings. Specifically, subfigures (a)~(h) correspond to various test samples: (a) is the normal instances; (b)~(d) refer the only one network-induced phenomenon; (e)~(g) holds two phenomena; (h) has all phenomena. Among them, the probability of problematic packets (*i.e.*, R_l, R_r, R_o) is either 0% or 20%. In subfigure (a), we find that the accuracy in LSTM is $\sim 99.25\%$, and ERNN realizes $98.47\% \sim 99.23\%$ ACC. As x tends to get smaller (up-to-bottom), ERNN exhibits some accuracy loss due to a higher probability of abnormal state transitions. Nonetheless, the performance drop of ERNN is less than 1% for the normal test set, which is trivial compared to the superiority of ERNN over LSTM in network-induced phenomena (introduced later). For the standard deviation, both LSTM and ERNN are basically $\leq 0.1\%$. From subfigures (b)~(d), it is clear that packet loss leads to the worst impact when there is only one phenomenon, *e.g.*, $\sim 46\%$ and $\sim 12\%$ accuracy loss in LSTM and ERNN. From subfigures (e)~(g), the results show that when appearing two problems, the performance decreases at most in packet loss and out-of-order are combined, *e.g.*, $\sim 48\%$ and $\sim 14\%$ accuracy loss in LSTM and ERNN. Those observations are the same as those in § 5.2. For instance, the worst case is not the scenario with all three phenomena existing, *i.e.*, subfigure (h); and packet loss and retransmission have a certain offsetting effect, *e.g.*, compare subfigures (b) and (e).

Overall, ERNN will introduce $< 1\%$ accuracy loss for the normal dataset, but could achieve the superiority of $5\% \sim 35\%$ ACC compared with LSTM. Meanwhile, ERNN possesses elasticity for different settings of the transition matrix. For example, subfigure (b) contains only packet loss. Even if we set about 13.3% probability for retransmission and out-of-order transition states respectively (*i.e.*, $x = y = 0.6$), ERNN still realizes $\sim 86.33\%$ ACC that outperforms LSTM by $\sim 33\%$. In all tests, the standard deviation of ERNN is always less than 3% while LSTM’s could reach $\sim 4\%$, which shows the stability of ERNN.

Interpret the Gating Mechanism. Based on the Integrated Gradients method, we generate the attribution map of model inference. The attribution matrix is equal to the input size and records the influence degree of each position in the input for the inference result. We average the attribution matrix of samples in the corresponding test scenarios and visualize them as a series of curves in Fig. 14.

From the line charts in Fig. 14, we find that ERNN is much less dependent on each packet than LSTM, especially in the

7. Since the initial state is normal and $x = 1.0$, the transition state is always normal no matter what y is. It represents the typical LSTM.

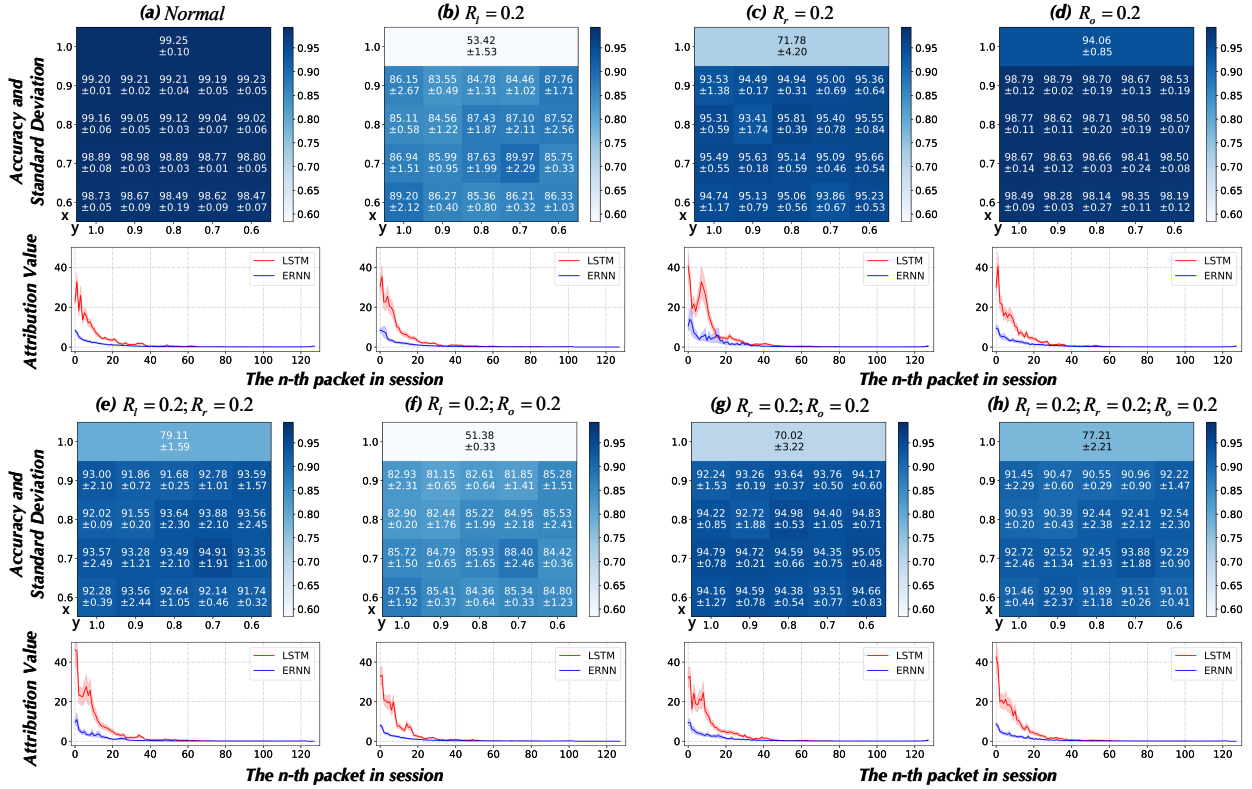


Fig. 14. The evaluation results and attribution maps of ERNN in different transition matrix settings.

first 20 packets, reducing $\sim \frac{3}{4}$ dependencies. In other words, LSTM relies heavily on some specific packet properties. When there are problems with the packets in these locations, the prediction results of the model will be greatly affected, *i.e.*, misclassifications. However, the error-resilient design in ERNN reduces the dependencies on local packets, instead focusing on the global semantics. We term this effect as *dependency dispersion* of model inference. Due to this effect, abnormal packets in some locations are less prone to cause model inference errors directly. From subfigures (a)~(h), we observe that the LSTM curve presents more oscillations, especially subfigure (c), which indicates that the dependencies are quite different at different positions. In contrast, the attribution value of ERNN tends to be smooth with the packet number. Combining the above experimental results and insights, we explain that as long as the gating mechanism is enabled with parameters in appropriate magnitude (*e.g.*, 5%~20%), it will essentially improve the model's error-resilient ability. And more exact transition probabilities will benefit fine-tuning the model performance. We subsequently discuss some real-world investigations to facilitate user selection of appropriate magnitudes.

To measure the model error of LSTM and ERNN, we also visualize their model decision boundary for in-depth analysis.⁸ A series of cases demonstrate ERNN maintains good error-resilient performance that manifested in the better generalization ability of classification boundaries.

Deployment Preparation. To advance the configuration of practitioners for ERNN in practice, we derive a series

of empirical observations based on the above experiments and investigate some real networks. In the experimental results of elasticity evaluations, all probability of the network-induced phenomena are set at 20%, and we assume that $x = y = 0.8$ as the expected benchmark, then summarize the following insights. (i) Under the same probability, the impact (lead to misclassification) of the three phenomena are loss > retransmission > out-of-order. This is consistent with the aforementioned evaluation results in § 5.2. According to the loss → retransmission → out-of-order, we analyze their parameter settings respectively. (ii) For packet loss, set a probability slightly higher than the benchmark, *e.g.*, compare with $x = y = 0.7$ and $x = y = 0.8$ in subfigures (b): 89.97% > 87.43%; (e): 94.91% > 93.64%; (f): 88.40% > 85.22%; (h): 93.88% > 92.44%. Note that we refer to the one with the greatest impact when multiple phenomena exist in the subfigure. (iii) For packet retransmission, set a probability to match the benchmark, *e.g.*, in subfigures (c): 95.81%; (g): 94.98%. (iv) For packet out-of-order, set a probability slightly lower than the benchmark, *e.g.*, compare with $x = y = 0.9$ and $x = y = 0.8$ in subfigures (d): 98.79% > 98.71%. Overall, for the same probability, we could set $R_l > R_r = \text{benchmark} > R_o$ given the extent of their influence on model inference.

TABLE 10
The network QoS in real-world scenarios (~100Mbps).

Application services				Home network			
Cross	Latency	R_l		Country	WiFi (LAN)	Mobile (4G)	
LAN	1~10ms	$\leq 0.5\%$			Latency	R_l	Latency
Cities	~60ms	$\leq 1\%$		US	~25ms	0.20%	~50ms 0.50%
Provinces	~100ms	$\leq 3\%$		UK	~10ms	0.20%	~25ms 0.20%
Operators	~150ms	$\leq 5\%$		Germany	~25ms	0.20%	~50ms 0.20%
Countries	~150ms	$\leq 20\%$		Japan	~10ms	0.10%	~50ms 0.20%

8. The analysis is detailed on https://github.com/TDSCCode/ERNN/tree/main/decision_boundary.

TABLE 11
The experimental results of LSTM and ERNN with different feature selections.

Intrusion Det.				LSTM										ERNN									
Phenomenon				fea ₁		fea ₂		fea ₃		fea ₄		fea ₅		fea ₁		fea ₂		fea ₃		fea ₄		fea ₅	
R _l	R _r	R _o		Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
0	0	0		99.1	99.2	99.4	99.3	99.5	99.4	99.5	99.4	99.5	99.4	99.0	99.1	99.2	99.3	99.5	99.4	99.4	99.4	99.4	99.2
0.2	0	0		55.0	55.6	64.5	64.6	69.5	71.1	73.7	73.4	75.7	75.8	86.1	85.2	89.7	89.6	92.5	92.4	93.9	93.3	94.0	93.9
0	0.2	0		72.8	72.4	80.4	80.6	82.4	82.5	83.6	83.5	85.7	86.0	93.5	93.2	94.8	94.9	97.6	97.6	97.0	96.7	97.2	97.5
0	0	0.2		94.7	95.2	96.5	96.4	97.1	97.2	97.2	97.3	97.2	97.2	98.1	98.1	98.1	98.1	98.2	98.1	99.2	99.2	99.1	99.1
0.2	0.2	0		78.4	78.5	81.7	81.9	85.6	85.7	86.3	86.2	91.1	91.3	93.0	92.5	94.2	94.6	96.2	96.2	96.3	96.5	96.0	96.1
0.2	0	0.2		51.4	51.2	57.6	57.6	63.2	63.9	68.9	69.4	73.5	73.5	81.1	80.1	83.9	82.7	90.5	90.6	88.1	87.6	93.7	93.0
0	0.2	0.2		70.6	70.8	78.2	78.6	80.2	81.2	82.6	82.1	83.6	83.3	91.5	91.5	92.5	92.5	93.5	93.5	94.7	94.4	95.6	95.7
0.2	0.2	0.2		77.0	77.4	81.3	81.9	83.4	83.4	85.9	85.4	89.1	89.2	90.3	89.7	91.0	90.0	93.0	93.1	94.5	94.1	95.3	95.4

Malware Det.				LSTM										ERNN									
Phenomenon				fea ₁		fea ₂		fea ₃		fea ₄		fea ₅		fea ₁		fea ₂		fea ₃		fea ₄		fea ₅	
R _l	R _r	R _o		Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
0	0	0		99.2	99.2	99.1	99.1	99.3	99.3	99.4	99.5	98.7	98.7	99.1	99.1	99.1	99.1	99.3	99.3	99.3	99.3	98.1	98.1
0.2	0	0		83.5	83.4	82.9	82.8	84.0	83.9	80.9	80.7	85.3	85.2	93.2	93.2	93.1	93.1	93.6	93.6	93.7	93.6	93.5	93.4
0	0.2	0		85.3	85.3	86.8	86.7	87.2	87.2	89.3	89.3	89.5	89.5	96.9	96.9	98.8	98.8	98.8	98.9	98.7	98.7	97.7	97.2
0	0	0.2		92.1	92.0	92.7	92.7	94.0	94.0	94.9	94.9	95.2	95.2	98.2	98.2	98.9	98.9	99.1	99.1	99.0	99.0	98.0	97.5
0.2	0.2	0		91.3	91.3	92.0	92.0	93.2	93.2	94.4	94.3	94.6	94.7	96.8	96.8	98.0	98.0	98.4	98.4	98.3	98.3	97.4	96.9
0.2	0	0.2		75.1	74.7	79.5	79.2	78.4	78.3	75.3	75.0	83.5	83.5	92.9	92.9	93.0	93.0	93.5	93.5	93.2	93.2	93.3	93.2
0	0.2	0.2		84.0	84.0	85.1	85.1	86.4	86.4	87.4	87.4	88.7	88.7	96.1	96.1	96.0	95.9	96.1	96.1	95.7	95.7	95.9	95.4
0.2	0.2	0.2		86.9	86.7	87.1	88.0	88.5	88.4	89.8	89.6	90.4	90.4	94.9	94.8	95.0	94.9	94.8	94.8	95.2	95.2	95.0	94.5

Then, we provide a case study for transition matrix calibration in real-world scenarios. By investigating network performance reports [59], [60], we summarize some network QoS from application services and the home networks of different countries in Table 10. It shows that network-induced phenomenon such as packet loss is relatively serious in cross-provinces and cross-countries scenarios, so robust detection methods are significant. Note that these QoS measurement reports mainly involve delay and packet loss, without counting retransmission and out-of-order. We assume that all three phenomena exist, the x, y could be set slightly less than $1 - R_l$, reference to Fig. 14 (h). Take cross-operators as an example, the packet loss R_l is $\sim 5\%$, we could set $x = y = 1 - 0.05 - 0.01(\text{fluctuation}) = 0.94$ and configure the following parameters.

$$F_m = \begin{pmatrix} 0.94\pm, & 0.02+, & 0.02\pm, & 0.02- \\ 0.94\pm, & 0.02+, & 0.02\pm, & 0.02- \\ 0.94\pm, & 0.02+, & 0.02\pm, & 0.02- \\ 0.94\pm, & 0.02+, & 0.02\pm, & 0.02- \end{pmatrix} \quad (11)$$

Among them, the probability of loss could be set slightly higher while out-of-order could be set lower. Noteworthy, we set the same probability (such as $(1 - y)/3$) for problematic packets for convenience of the experiments, while they can be adjusted as different values in practice. Furthermore, according to the specific network environment, users can utilize tools such as iperf, X-Launch, and ATTKPING to count and calculate the latency, packet loss rate, out-of-order, and other network metrics.

7.2 Feature Selection

Considering that only using the packet length could be constrained to deal with more sophisticated traffic analysis tasks [34], we conduct additional experiments for feature selection. Our extended feature vector includes five parts: (i) dim_0 : the packet length in the session; (ii) dim_1 : the timestamp delta (s) in the session; (iii) dim_2 : the "time to live" field in the IP layer; (iv) dim_3 : the sum of TCP flags (i.e., ACK, URG, PSH, RST, SYN, FIN); (v) dim_4 : the "window size value" field in the TCP layer. For the 5-dimensional feature vector, we select $\{[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]\}$ (denote $fea_1, fea_2, fea_3, fea_4, fea_5$) to evaluation, respectively.

The results of intrusion detection and malware detection are summarized in Table 11, where we set $x = y = 0.8$ for ERNN. Overall, ERNN clearly outperforms LSTM when exists problematic packets, whichever feature set is chosen. For the normal test data, the accuracy loss of ERNN is insignificant (less than 0.6%), even the performance of ERNN is competitive, e.g., using fea_3 . For the feature selection in ERNN, fea_3, fea_4 , and fea_5 perform superior performance in most cases, e.g., ERNN realizes fea_3 : 99.5%, fea_4 : 99.4%, and fea_5 : 99.4% ACC in intrusion detection (normal test data). Between fea_3, fea_4 , and fea_5 , their effects are comparable, and neither side has an overwhelming advantage. For instance, the highest (red mark) ACC of intrusion detection in ERNN with $R_l = 0.2$ is fea_5 : 94.0%; $R_r = 0.2$ is fea_3 : 97.6%; $R_o = 0.2$ is fea_4 : 99.2%. By counting the best performance (red mark) in each row, we find that LSTM uses fea_5 to perform the best while ERNN utilizes fea_3 . This indicates that increasing the feature dimension can alleviate the impact of the problem packets on LSTM to a certain extent. In addition, the impact of network-induced phenomena on models in multi-classification tasks (intrusion detection) is more serious than binary classification (malware detection). We continue using the attribution map for analysis.

Fig. 15 shows the attribution matrices (after normalization) of LSTM and ERNN in intrusion detection and malware detection dataset⁹. With the feature set increasing (top-to-bottom), the dependencies of LSTM transition from single-row to multi-row. We could understand this process as the *vertical dispersion* of inference dependence in subfigures (a) and (c). More specifically, LSTM relies heavily on the attributes of several packets when holds a low-dimension feature. Once there is a problem (e.g., packet loss) with the packets in those locations, the inference results will be seriously affected. However, when LSTM learns high-dimensional feature vectors, the position of dependencies may not be the same for each dimension, e.g., in the bottom of subfigure (c), LSTM relies heavily on the 15-th~20-th packets for dim_1 but 3-th~5-th packets for dim_3 . This brings some

9. The attribution matrix is calculated as the mean of 8 test data, including the normal test set and the 7 combinations for the network-induced phenomenon.

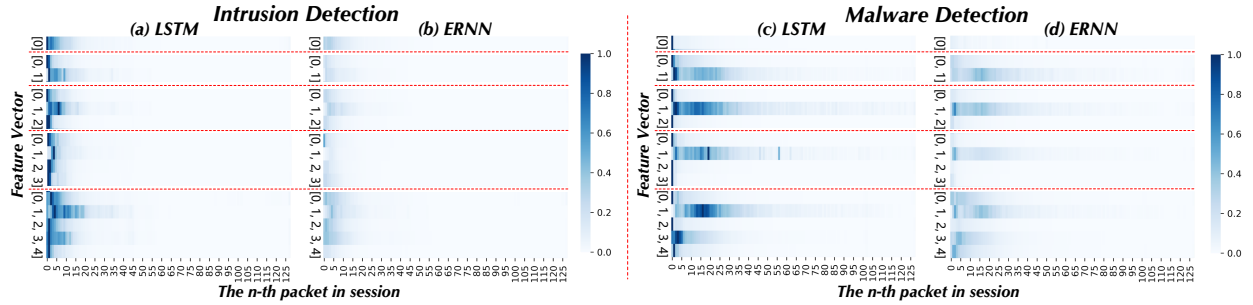


Fig. 15. The attribution matrices visualization of LSTM and ERNN based on different feature selections.

mitigating effects to the loss of accuracy, since these key positions may not have problems at the same time.

Looking back and examining ERNN in subfigures (b) and (d), its effects can be described as *lateral dependency dispersion*. No matter which feature vector is used, it is clear that ERNN is much less dependent (behaves as a lighter color) on each packet than LSTM. More importantly, the dependency dispersion effect of ERNN is more significant than increasing the feature dimension, thus keeping the classifier to make decisions accurately when some features are changed by the network-induced phenomenon. In conclusion, ERNN could throughout perform remarkable error-resilient capability regardless of using which feature set. Meanwhile, appropriately enriching the feature set could also improve the detection performance of ERNN such as the case of fea_3 . Interestingly, when choosing different feature sets for ERNN, the accuracy of normal testing data and abnormal ones are generally homogeneous. It could benefit to determine a suitable feature set for practitioners in practice.

7.3 Combine with Traffic Normalization Tools

As § 2.4 stated, the existing tools could perform traffic normalization mainly involving rearranging out-of-order packets and discarding retransmission packets for TCP-based protocol. This section will investigate the quality of the dataset and conduct combination experiments with the normalization tool. We use *tshark* to fix traffic, including retransmission and out-of-order. The results are summarized in Table 12. It is clear that network-induced phenomena of attack samples tend to be rare in IDS2017 & IDS2018. Among them, the retransmission ratios are $\sim 0.01\%$, and the out-of-order ratios are generally less than 0.001% . For the benign samples, the network-induced phenomena are slightly more, *i.e.*, $R_r=1.18\%$ and $R_o=0.361\%$. In addition, the problematic packet ratio from the malware dataset ($R_r=\sim 2\%$ and $R_o=\sim 0.6\%$) is of the same magnitude as benign instances of the IDS dataset. Overall, it indeed exists network-induced phenomena in the training set, but there are still some gaps compared with the complex network environment, *e.g.*, the retransmission and out-of-order ratios could reach $\sim 20\%$ in the aforementioned experiment. More importantly, the problematic packets in the dataset are mainly involved in benign samples, which also leads to the fact that the attack flow may be misclassified by the model if there is a slight change. We visualize the model decision boundaries of LSTM and ERNN in § 7.1. The results show that when there are network-induced phenomena, most attack instances are

indeed misclassified by LSTM, while ERNN performs robust performance and error-resilient ability.

TABLE 12

The retransmission and out-of-order ratios are measured using *tshark* in training set from IDS and malware. Particularly, DoS&DDoS-based and Brute Force-based attacks will generate massive retransmission packets, which do not belong to network-induced phenomena.

IDS2017 & IDS2018					
Type	R_r (%)	R_o (%)	Type	R_r (%)	R_o (%)
Benign	1.180	0.361	FTP-Patator	-	0.000
Botnet	0.012	0.000	SSH-Patator	-	0.000
DDoS	-	0.000	Infiltration	0.025	0.000
DoS-GoldenEye	-	0.000	PortScan	0.000	0.000
DoS-Hulk	-	0.024	Web-attack	0.003	0.000
DoS-Slowhttptest	-	0.000	Heartbleed	0.004	0.000
DoS-Slowloris	-	0.000			
CICAndMal2017					
Type	Benign	Adware	Ransomware	Scareware	SMS Mal.
R_r (%)	1.660	2.407	1.929	2.129	2.240
R_o (%)	0.369	0.607	0.334	0.449	0.333

TABLE 13

The detection accuracy when using *tshark* normalization tool.

IDS	Offline testing set				$N = 7$ in parallel topology			
	None	Train	Test	Tra&Tes	None	Train	Test	Tra&Tes
FS-Net	99.07	98.23	98.56	99.27	94.44	91.52	94.98	95.40
LSTM	99.32	98.15	98.58	99.34	93.46	90.68	94.71	95.27
ERNN	99.24	99.31	99.28	99.33	98.87	98.88	98.83	98.96
Mal	Offline testing set				$X = 25$ in multi-user scenario			
	None	Train	Test	Tra&Tes	None	Train	Test	Tra&Tes
Whisper	99.32	98.15	99.06	99.34	93.51	91.38	93.55	93.79
LSTM	99.19	97.76	98.65	99.28	89.92	87.36	91.94	92.81
ERNN	99.13	99.23	99.19	99.28	97.23	97.31	97.25	97.36

Moreover, we combine *tshark* with models to evaluate their performance. For both IDS and malware detection, the training sets come from the datasets, one group of the testing set comes from the datasets, and one group comes from our testbed that exists network-induced phenomena. From the left part of Table 13, we observe that baseline models (*i.e.*, FS-Net and LSTM in IDS; Whisper and LSTM in malware detection) will present accuracy loss if solely using *tshark* in the training set or the testing set. We suspect that this is due to using *tshark* solely on one party, introducing some heterogeneity between the training and testing sets. For the right part of Table 13 (the testing set is from our testbed), we find that using *tshark* in the training set will cause a drop in the accuracy of baselines, yet using *tshark* in the testing set could improve their performance. This shows that using traffic normalization tools can indeed benefit the detection for testing data with network-induced phenomena. As expected, all models achieve the best results when using *tshark* both on

the training and testing sets. For the scenarios with network-induced phenomena, the traffic normalization tool could bring certain improvements for FS-Net and LSTM, while it does not help much for Whisper and ERNN. This may be attributed to the tool could not fix lost packets. Overall, ERNN still shows remarkable error-resilient performance, *i.e.*, ERNN presents 99.24% $\rightarrow$ 98.96% in IDS and 99.13% $\rightarrow$ 97.36% in malware detection. The accuracy drops of ERNN are less than 1/3 of baseline models.

8 DISCUSSION

Practicality of ERNN. As packet loss, retransmission, and out-of-order are common network-induced phenomena, the error-resilient capability of ERNN can be widely adopted in diverse network scenarios. For instance, Autonomous Systems (ASes) are small network units that can independently decide which routing protocol to use. ERNN can benefit encrypted traffic detection models deployed in ASes routers. Furthermore, ERNN can also be applied to cloud-based solutions where the traffic processing units are typically deployed at Cloud Security Service Providers (CSSPs) or Internet Service Providers (ISPs). Specifically, ERNN can support encrypted traffic detection of flow collection nodes in ISPs, pursuing high accuracy without reordering the packets.

Other Application Scenarios. In § 3, we mainly consider packet loss, retransmission, and out-of-order. Meanwhile, ERNN can also be adapted to other network-induced phenomena. For instance, the repacketized retransmission [29] refers to a case that the sender can reassemble the packet to transmit more bytes (*e.g.*, MTU) when the TCP server finds an error or does not receive a response packet. This process can be approximated by the multiple “shape” actions of the session gate. Furthermore, ERNN’s design can benefit traffic detection tasks to mitigate the influence of malicious manipulation [33].

Moreover, we list some other usage scenarios here in addition to encryption network behaviors identification. (i) *Covert channel detection.* Capturing packet distributions makes it possible to detect covert channels, thereby providing a valuable asset for cyber-forensic investigations [46]. (ii) *Website fingerprinting.* Privacy-enhancing technologies like OpenSSL or OpenVPN enable users to hide the destination address behind a proxy and the content of website visits from external observers. However, it is possible to identify which sites users accessed by ML classification algorithms [61]. (iii) *Botnet detection.* Botnets jeopardize the security of multiple organizations, emerging as a highly profitable activity for malicious operators. Meanwhile, current approaches can identify botnet traffic through the analysis of packet length distributions of network hosts [62]. ERNN can benefit these tasks that classify specific target flows.

Robustness against the Adversarial Attacks. As a machine learning model, ERNN could be vulnerable to adversarial manipulations. First, the typical adversarial perturbation schemes refer to tampering with some location of the input features in white-box settings or black-box settings, *e.g.*, modifying the pixels in an image to make model misclassification. This perturbation is somewhat similar to our shape design (action 4) for the input in ERNN. In other words, the dependency dispersion effect of

ERNN is the potential to alleviate the impact of adversarial attacks. Moreover, tampering with traffic is not arbitrarily available given the need to comply with protocol standards and guarantee attack effectiveness. Recent researches [21], [63] show that using Selective Symbolic Execution (S2E) can automatically mine the available adversarial instances, including *evasion* and *insertion* packets. So-called evasion and insertion packets refer to the different packet sequences between the middlebox and end hosts due to discrepancies in protocol implementations, *e.g.*, DPI middleboxes usually implement simplified state machines. In principle, they have great homogeneity with packet loss and retransmission. Overall, there are some similarities between network-induced phenomena and popular adversarial attack mechanisms, and ERNN could be further explored for this problem.

Data Augmentation. The data augmentation research [64] generally aims to improve model robustness by introducing a series of noises. Some representative work uses traditional transformations [65] (such as cropping, rotating, and flipping), random erasing [66], generative adversarial network [67], and so on. Most of the existing data augmentation work is for image and text data, which could not directly cope with network-induced phenomena for the following reasons. For one thing, packet loss, retransmission, and out-of-order are specific problems in the network domain, which are different from image cropping, rotating, and flipping. For another, network-induced phenomena usually have some temporal properties, which is also the original intention of our design transition matrix in the gating mechanism, *e.g.*, the probability that the next packet will be retransmitted tends to be different when the current packet is lost or normal. However, the existing data augmentation methods lack consideration for the contextual semantics of temporal information.

Limitations and Future Works. Our work has a few limitations. First, although ERNN can greatly alleviate the impact of network-induced phenomena, setting different parameters still causes a small fraction of the accuracy gap. We tend to work extensively with more security vendors in the future to refinement configuration empirical to improve applicability. Then, more network QoS metrics can be considered in ERNN, we would like to integrate ERNN with some network measurement tools so that further enrich the functionality. Finally, implementing customized solutions is also a promising direction given user diversity in the real world. As part of future work, we will advance a series of schemes for customized adaptation. For example, the classifier trained from a public dataset could be treated as the pre-trained model, then fine-tuned with few-shot samples (which may reflect real-world network-induced phenomena) from the practical environment.

9 CONCLUSION

In this paper, we propose ERNN, an error-resilient RNN model that is specially designed for network-induced phenomena. Through the four types of actions and Mealy machine update states in session gate, ERNN realizes the state-of-the-art encrypted traffic detection against common network-induced phenomena. We implement ERNN and evaluate it extensively on both wired and wireless testbeds.

By practical evaluation with dynamic bandwidth utilization and different network topologies, we demonstrate that ERNN can still identify 98.63% of encrypted intrusion traffic when facing about 16% abnormal packet sequences on a 10 Gbps wired dataplane. Similarly, in wireless scenarios with different degrees of abnormal packet sequences caused by multi-user concurrency, ERNN can still robustly identify more than 97% of the encrypted malware traffic. Meanwhile, ERNN possesses superior stability and scalability in terms of parameter settings and feature selection. Finally, we interpret and visualize the *dependency dispersion* effect of ERNN.

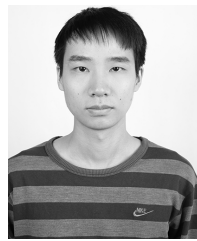
ACKNOWLEDGMENTS

This work was supported in part by National Natural Science Foundation of China (62227805, 62072398), by National Key R&D Program of China (2020AAA0107700), by Alibaba-Zhejiang University Joint Institute of Frontier Technologies, by Zhejiang Key R&D Plan (2021C01116), by Leading Innovative and Entrepreneur Team Introduction Program of Zhejiang (2018R01005), by National Key R&D Program of China (2021YFB2700603), by National Natural Science Foundation of China (62172405), and by Research Institute of Cyberspace Governance in Zhejiang University, by National Key Laboratory of Science and Technology on Information System Security, by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Key Laboratory of Cyberspace Situation Awareness of Henan Province.

REFERENCES

- [1] X. Li *et al.*, "Online internet anomaly detection with high accuracy: A fast tensor factorization solution," in *INFOCOM*. IEEE, 2019, pp. 1900–1908.
- [2] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *INFOCOM*, 2021.
- [3] M. Zhang *et al.*, "Poseidon: Mitigating volumetric ddos attacks with programmable switches," in *NDSS*. The Internet Society, 2020.
- [4] H. Li *et al.*, "vnids: Towards elastic security with safe and efficient virtualization of network intrusion detection systems," in *CCS*. ACM, 2018, pp. 17–34.
- [5] Z. Liu *et al.*, "Middlepolice: Toward enforcing destination-defined policies in the middle of the internet," in *CCS*. ACM, 2016, pp. 1268–1279.
- [6] Q. Kang *et al.*, "Programmable in-network security for context-aware BYOD policies," in *USENIX Security Symposium*. USENIX Association, 2020, pp. 595–612.
- [7] J. Xing, W. Wu, and A. Chen, "Ripple: A programmable, decentralized link-flooding defense against adaptive adversaries," in *USENIX Security Symposium*. USENIX Association, 2021.
- [8] J. Ning, G. S. Poh *et al.*, "Privdpi: Privacy-preserving encrypted traffic inspection with reusable obfuscated rules," in *CCS*. ACM, 2019, pp. 1657–1670.
- [9] M. Korczynski and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *INFOCOM*. IEEE, 2014, pp. 781–789.
- [10] C. Liu *et al.*, "Mampf: Encrypted traffic classification based on multi-attribute markov probability fingerprints," in *IWQoS*. IEEE, 2018, pp. 1–10.
- [11] M. Shen *et al.*, "Certificate-aware encrypted traffic classification using second-order markov chain," in *IWQoS*. IEEE, 2016.
- [12] M. Shen, M. Wei *et al.*, "Classification of encrypted traffic with second-order markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 8, pp. 1830–1843, 2017.
- [13] C. Liu *et al.*, "Fs-net: A flow sequence network for encrypted traffic classification," in *INFOCOM*. IEEE, 2019, pp. 1171–1179.
- [14] K. Bartos *et al.*, "Optimized invariant representation of network traffic for detecting unseen malware variants," in *USENIX Security Symposium*. USENIX Association, 2016.
- [15] Y. Mirsky *et al.*, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *NDSS*. The Internet Society, 2018.
- [16] B. Merino, *Instant traffic analysis with Tshark how-to*. Packt Publishing Ltd, 2013.
- [17] M. A. Jamshed *et al.*, "mos: A reusable networking stack for flow monitoring middleboxes," in *NSDI*. USENIX Association, 2017, pp. 113–129.
- [18] Y. Mao *et al.*, "Mencius: Building efficient replicated state machine for wans," in *OSDI*. USENIX Association, 2008, pp. 369–384.
- [19] L. Liu *et al.*, "Modelling and estimation for uncertain systems with transmission delays, packet dropouts, and out-of-order packets," *Complex.*, vol. 2018, pp. 7531547:1–15, 2018.
- [20] M. Vutukuru *et al.*, "Efficient and robust TCP stream normalization," in *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2008, pp. 96–110.
- [21] Z. Wang *et al.*, "Symtcp: Eluding stateful deep packet inspection with automated discrepancy discovery," in *NDSS*. The Internet Society, 2020.
- [22] Y. Go *et al.*, "Gaining control of cellular traffic accounting by spurious TCP retransmission," in *NDSS*. The Internet Society, 2014.
- [23] A. Kangarlou *et al.*, "Chronicle: Capture and analysis of NFS workloads at line rate," in *FAST*. USENIX Association, 2015, pp. 345–358.
- [24] X. Luo *et al.*, "Design and implementation of TCP data probes for reliable and metric-rich network path monitoring," in *USENIX Annual Technical Conference*. USENIX Association, 2009.
- [25] S. Kumar *et al.*, "Performant TCP for low-power wireless networks," in *NSDI*. USENIX Association, 2020, pp. 911–932.
- [26] Y. Lu *et al.*, "Multi-path transport for RDMA in datacenters," in *NSDI*. USENIX Association, 2018, pp. 357–371.
- [27] M. T. Arashloo *et al.*, "Enabling programmable transport protocols in high-speed nics," in *NSDI*. USENIX Association, 2020, pp. 93–109.
- [28] P. Jin *et al.*, "Postman: Rapidly mitigating bursty traffic by offloading packet processing," in *USENIX Annual Technical Conference*. USENIX Association, 2019.
- [29] S. Lee *et al.*, "Packet out-of-order and retransmission in statistics-based traffic analysis," in *APNOMS*, 2014.
- [30] K. He *et al.*, "Presto: Edge-based load balancing for fast datacenter networks," in *SIGCOMM*. ACM, 2015, pp. 465–478.
- [31] C. Pascoal *et al.*, "Robust feature selection and robust PCA for internet traffic anomaly detection," in *INFOCOM*. IEEE, 2012.
- [32] G. Xie *et al.*, "Subflow: Towards practical flow-level traffic classification," in *INFOCOM*. IEEE, 2012, pp. 2541–2545.
- [33] Y. Li *et al.*, "Robust Online Learning against Malicious Manipulation with Application to Network Flow Classification," in *INFOCOM*. IEEE, 2021.
- [34] S. Zhu *et al.*, "You do (not) belong here: detecting DPI evasion attacks with context learning," in *CoNEXT*. ACM, 2020, pp. 183–197.
- [35] S. Li *et al.*, "Thunderbolt: Throughput-optimized, quality-of-service-aware power capping at scale," in *OSDI*. USENIX Association, 2020, pp. 1241–1255.
- [36] G. Apostolopoulos *et al.*, "Quality of service based routing: A performance perspective," in *SIGCOMM*. ACM, 1998, pp. 17–28.
- [37] M. Piatek *et al.*, "Contracts: Practical contribution incentives for P2P live streaming," in *NSDI*. USENIX Association, 2010, pp. 81–94.
- [38] Q. Zhang, Y. Ma, J. Wang, and X. Li, "UDP traffic classification using most distinguished port," in *APNOMS*. IEEE, 2014, pp. 1–4.
- [39] J. Park, S. Yoon, and M. Kim, "Performance improvement of payload signature-based traffic classification system using application traffic temporal locality," in *APNOMS*. IEEE, 2013, pp. 1–6.
- [40] S. Sen, O. Spatscheck, and D. Wang, "Accurate, scalable in-network identification of p2p traffic using application signatures," in *WWW*. ACM, 2004, pp. 512–521.
- [41] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 1, pp. 63–78, 2018.
- [42] A. Saha *et al.*, "Learning network traffic dynamics using temporal point process," in *INFOCOM*. IEEE, 2019, pp. 1927–1935.
- [43] S. Al-Riyami *et al.*, "A re-evaluation of intrusion detection accuracy: Alternative evaluation strategy," in *CCS*. ACM, 2018, pp. 2195–2197.
- [44] C. Fu *et al.*, "Realtime Robust Malicious Traffic Detection via Frequency Domain Analysis," in *CCS*. ACM, 2021, pp. 3431–3446.

- [45] S. Kumar *et al.*, "Bringing cross-layer MIMO to today's wireless lans," in *SIGCOMM*. ACM, 2013, pp. 387–398.
- [46] D. Barradas *et al.*, "FlowLens: Enabling Efficient Flow Classification for ML-based Network Security Applications," 2021.
- [47] K. P. Dyer, S. E. Coull *et al.*, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2012, pp. 332–346.
- [48] Intel, "Data Plane Development Kit," 2014.
- [49] R. Miao *et al.*, "Silkroad: Making stateful layer-4 load balancing fast and cheap using switching asics," in *SIGCOMM*. ACM, 2017, pp. 15–28.
- [50] C. I. for Cybersecurity, "Intrusion detectionevaluation dataset (cicids2017)." [EB/OL], 2018, <https://www.unb.ca/cic/datasets/ids-2017.html> Accessed November 27, 2020.
- [51] C. C. Institute, "Cse-cic-ids2018 on aws." [EB/OL], 2018, <https://www.unb.ca/cic/datasets/ids-2018.html>.
- [52] P. M. Comar *et al.*, "Combining supervised and unsupervised learning for zero-day malware detection," in *INFOCOM*. IEEE, 2013, pp. 2022–2030.
- [53] K. Bartos *et al.*, "Optimized invariant representation of network traffic for detecting unseen malware variants," in *USENIX Security Symposium*. USENIX Association, 2016.
- [54] <https://developer.android.com/studio/run/emulator.html>.
- [55] C. I. for Cybersecurity, "Android malware dataset (cic-andmal2017)." [EB/OL], 2018, <https://www.unb.ca/cic/datasets/andmal2017.html>, 2018.
- [56] L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. Nov, 2008.
- [57] M. Sundararajan *et al.*, "Axiomatic attribution for deep networks," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 3319–3328.
- [58] "Captum: Model Interpretability for PyTorch," <https://captum.ai/>.
- [59] Codavel, "Real-world latency and packet loss." [EB/OL], <https://blog.codavel.com/performance-report-defining-use-cases>.
- [60] Raysync, "Transmission protocol technical white paper." [EB/OL], 2022, <https://www.raysync.cn/support/document>.
- [61] D. Barradas *et al.*, "Effective detection of multimedia protocol tunneling using machine learning," in *USENIX Security Symposium*. USENIX Association, 2018.
- [62] P. Narang *et al.*, "Peershark: Detecting peer-to-peer botnets by tracking conversations," in *IEEE Symposium on Security and Privacy Workshops*. IEEE Computer Society, 2014, pp. 108–115.
- [63] Z. Wang *et al.*, "Themis: Ambiguity-aware network intrusion detection based on symbolic model comparison," in *CCS*. ACM, 2021, pp. 3384–3399.
- [64] Q. Xie *et al.*, "Unsupervised data augmentation for consistency training," in *NeurIPS*, 2020.
- [65] L. Perez *et al.*, "The effectiveness of data augmentation in image classification using deep learning," *CoRR*, vol. abs/1712.04621, 2017.
- [66] Z. Zhong *et al.*, "Random erasing data augmentation," in *AAAI*. AAAI Press, 2020, pp. 13 001–13 008.
- [67] T. Tran *et al.*, "A bayesian data augmentation approach for learning deep models," in *NIPS*, 2017, pp. 2797–2806.



Jialun Jiang is a master student at Zhejiang University, Hangzhou, China. His research interests include machine learning, traffic identification, and privacy-preserving.



Fengyuan Yu His research interests include machine learning, traffic identification, and privacy-preserving. is a undergraduate student at Computer Science in College of Computer Science and Technology, Zhejiang University, Hangzhou, China. His research interests include machine learning and traffic identification.



Fan Zhang (Member, IEEE) received his Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, CT, USA, in 2011. He is currently a Full Professor with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China, and also with the Alibaba–Zhejiang University Joint Institute of Frontier Technologies, Hangzhou. His research interests include system security, hardware security, network security, cryptography, and computer architecture.



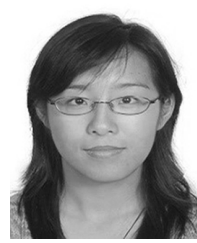
Congyuan Xu received the Ph.D. degree in Electronics Science and Technology from Zhejiang University, Hangzhou, China in 2019, and the B.S. degree in Applied Physics from Xidian University, Xi'an, China, in 2013. Currently, he is a lecturer in the College of Information Science and Engineering, Jiaxing University, Jiaxing, China. His research interests focus on machine intelligence and cyberspace security.



Xinjie Zhao was born in 1986. He received the B.S., M.S., and Ph.D. degrees from Ordnance Engineering College, in 2006, 2009, and 2012, respectively. He is currently with the Institute of North Electronic Equipment. His main research interest includes side channel analysis, fault analysis, and combined analysis in cryptography. He won the best paper award in COSADE 2012 and the Outstanding Doctoral Dissertation Award in the province of Hebei.



Ziming Zhao is a Ph.D. candidate at Zhejiang University, Hangzhou, China. His research interests include machine learning, traffic identification, AI security, and privacy-preserving.



Rui Zhang is an associate researcher with the SKLOIS, Institute of Information Engineering, CAS, China. She was a post-doctor in Institute of Software, CAS. She was a visiting scholar in Georgia Institute of Technology from 2009 to 2010 and 2018 to 2019. She has published more than 40 technical papers in international journals and conference proceedings. Her research interests include blockchain security, security protocol and applied cryptography.



Zhaoxuan Li is a Ph.D. candidate at State Key Laboratory of Information Security (SKLOIS) with Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), Beijing, China. His research interests include artificial intelligence, blockchain security, and privacy-preserving.



Shize Guo was born in 1969. He received the Ph.D. degree from the Harbin Institute of Technology, in 1989, and the M.S. and B.S. degrees from Ordnance Engineering College, China, in 1991 and 1988, respectively. He is currently a Researcher with the Institute of North Electronic Equipment and also a Professor with the Beijing University of Post and Telecommunications. His main research interest includes information technology and information security.